

04_Zomato

July 11, 2025

```
[350]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[351]: df1 = pd.read_csv('zomato.csv')
```

```
[352]: df1.head() # collection of restaurant
```

```
[352]:
```

	url \	address	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	942, 21st Main Road, 2nd Stage, Banashankari, ...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	1112, Next to KIMS Medical College, 17th Cross...	San Churro Cafe
3	https://www.zomato.com/bangalore/addhuri-udupi...	1st Floor, Annakuteera, 3rd Stage, Banashankar...	Addhuri Udupi Bhojana
4	https://www.zomato.com/bangalore/grand-village...	10, 3rd Floor, Lakshmi Associates, Gandhi Baza...	Grand Village

	online_order	book_table	rate	votes	phone \
0	Yes	Yes	4.1/5	775	080 42297555\r\n+91 9743772233
1	Yes	No	4.1/5	787	080 41714161
2	Yes	No	3.8/5	918	+91 9663487993
3	No	No	3.7/5	88	+91 9620009302
4	No	No	3.8/5	166	+91 8026612447\r\n+91 9901210005

	location	rest_type \
0	Banashankari	Casual Dining
1	Banashankari	Casual Dining
2	Banashankari	Cafe, Casual Dining
3	Banashankari	Quick Bites
4	Basavanagudi	Casual Dining

```

                                dish_liked \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                                Panipuri, Gol Gappe

                                cuisines approx_cost(for two people) \
0  North Indian, Mughlai, Chinese                        800
1      Chinese, North Indian, Thai                      800
2          Cafe, Mexican, Italian                      800
3      South Indian, North Indian                      300
4      North Indian, Rajasthani                        600

                                reviews_list menu_item \
0  [('Rated 4.0', 'RATED\n A beautiful place to ...    []
1  [('Rated 4.0', 'RATED\n Had been here for din...    []
2  [('Rated 3.0', 'RATED\n Ambience is not that ...    []
3  [('Rated 4.0', 'RATED\n Great food and proper...    []
4  [('Rated 4.0', 'RATED\n Very good restaurant ...    []

    listed_in(type) listed_in(city)
0          Buffet   Banashankari
1          Buffet   Banashankari
2          Buffet   Banashankari
3          Buffet   Banashankari
4          Buffet   Banashankari

```

```
[353]: df1['menu_item'].value_counts() # observation: there are many as (39617) empty
```

```
[353]: []
39617
['Butter Chicken Pizza', 'Bombay Veggie Burger', 'Tawa Paneer Burger', 'Oh So
Cheesy Burger', 'Barbecue Chicken Burger', 'Peri Peri Chicken Burger', 'BBQ
Lamb', 'Cheesy Chicken', 'Crunchy Ferrero', 'Lots of Nuts', 'Kadhai Paneer
Pizza', 'Chilli Paneer and Mushroom Pizza', 'Chipotle Veggie Pizza', 'Creamy
Cheese Pizza', 'Southern Veggie Korma Pizza', 'Popeye, The Corny Man Pizza',
'Paneer Tikka Pizza', 'Desi Margherita Pizza', 'Roasted Paneer with Mustard
Pizza', 'Bangalore Express Pizza', 'Italian Chaska Pizza', 'Mayo and Cheese
Pizza', 'Classic Margherita', 'Cajun Hawaiian Pizza', 'Butter Chicken Pizza',
'Lasooni Bhuna Murg Pizza', 'Chilly Chicken Pizza', 'Hot Garlic Chicken Pizza',
'Murg Barbecue Pizza', 'Mediterranean Mutton Keema Pizza', 'Super Green Burger',
'Quinoa Black Bean Burger', 'Bombay Veggie Burger', 'Tawa Paneer Burger', 'Oh So
Cheesy Burger', 'Barbecue Chicken Burger', 'Chipotle Lamb Burger', 'Crumb Fried
Fish Burger', 'Peri Peri Chicken Burger', 'BBQ Lamb', 'Cheesy Chicken', 'Picnic
Chicken Burger', 'Korean Grilled Chicken with Kimchi', 'Kiddy Kat Strawberry',
'Kiddy Kat Chocolate', 'Bubblemum', 'Oreo Shake', 'Caramel Shake', 'Chocolate

```

Shake', 'Strawberry Shake', 'Vanilla Shake', 'Cold Coffee', 'Virgin Mojito', 'Iced Tea', 'Hot Chocolate', 'Cappuccino', 'Latte', 'Espresso Shot', 'Brownie Shake', 'Red Velvet Cheesecake Shake', 'Oreo Cheesecake Shake', 'Salted Caramel Popcorn', 'Coconut Crumble', 'Coffee Crunch', 'Bira', 'Thanda Paan', 'Sticky Toffee Pudding', 'Blueberry Cheesecake', 'Nutella Cheesecake', 'Caramel Custard with Strawberry or Figs', 'Red Velvet Cheesecake', 'Tiramisu', 'Zesty Vanilla and Topsy Brownie', 'Oreo Explosion', 'Chocolate Almond Crumble', 'Crunchy Ferrero', 'Stoner's Chocolate Decadence', 'Brownie Bash', 'Chocolate Lava', 'Dark Chocolate Caramel', 'Chocolate Overload', 'Lots of Nuts', 'Peanut Butter Crunch', 'Nutty Fudgy', 'Lemon Pistachio Biscotti', 'Frutiticious Ice Cream', 'Strawberry Tease Ice Cream', 'Mango Tango Ice Cream', 'Lychee Lovers Ice Cream', 'Monkey Business Ice Cream', 'Apple Crumble Ice Cream', 'Blueberry Bliss Ice Cream']

11

['Avil Milk', 'Oreo Shake', 'Chocolate Shake', 'Royal Falooda', 'Fruits Salad with Ice Cream', 'Fruits Salad with Ice Cream', 'Blackcurrant Ice Cream', 'Spanish Delight Ice Cream', 'Chillout Special Ice Cream', 'Chocolate Ice Cream', 'Butterscotch Ice Cream', 'Pista Ice Cream', 'Strawberry Ice Cream', 'Vanilla Ice Cream', 'Royal Falooda', 'Strawberry Falooda', 'Chillout Special Falooda', 'Lemon Juice', 'Mint Lemon Juice', 'Ginger Lemon Juice', 'Lemon Soda', 'Mint Lemon Soda', 'CAP Juice', 'Kiwi Juice', 'Carrot Juice', 'Pomegranate Juice', 'Watermelon Juice', 'Muskmelon Juice', 'Anjeer Juice', 'Pure Strawberry Juice', 'Papaya Juice', 'Orange Juice', 'Mosambi Juice', 'Grape Juice', 'Apple Juice', 'Pineapple Juice', 'Banana Juice', 'Rose Milk', 'Chikoo Shake', 'Sharja Shake', 'Kiwi Shake', 'Oreo Shake', 'Raspberry Shake', 'Pomegranate Shake', 'Mango Shake', 'Muskmelon Shake', 'Papaya Shake', 'Apple Shake', 'Berries Shake', 'Spanish Delight Shake', 'Blackcurrant Shake', 'Chillout Special Shake', 'Chocolate Shake', 'Butterscotch Shake', 'Strawberry Shake', 'Vanilla Shake', 'Pista Shake', 'Cold Coffee Shake', 'Avil Milk', 'Special Milk', 'Kulki Sarbath', 'Sweet Lassi', 'Chocolate Lassi', 'Mango Lassi', 'Banana Lassi', 'Strawberry Lassi']

10

['Chicken Cheese Burger', 'Chicken Billys BIG Burger', 'French Fries', 'Alfredo Veg Pasta', 'Brownie Waffles', 'Veg Burger', 'Veg Cheese Burger', 'Veg Caramelized onion Burger', 'Mushroom Classic Burger', 'Veg English Cheddar Cheese Burger', 'Veg Chipotle Corn Burger', 'Veg Billys BIG Burger', 'Chicken Burger', 'Chicken Cheese Burger', 'Chicken Caramelized Onion Burger', 'Chicken Mushroom Burger', 'Chicken English Cheddar Cheese Burger', 'Chicken Chipotle Corn Burger', 'Chicken Billys BIG Burger', 'Chicken Nuggets Burger', 'Veg Grilled Sandwich', 'Paneer Burji Sandwich', 'Toasted Paneer Sandwich', 'Grilled Mushroom Sandwich', 'Corn and Cheese Sandwich', 'Egg Sandwich', 'Egg Chutney Sandwich', 'Chicken Mayo Sandwich', 'Chicken Cheese Sandwich', 'Hotdog Sandwich', 'Tuna Sandwich', 'Tangy Tomato Pasta', 'Spicy Penne Pasta', 'Alfredo Veg Pasta', 'Smoked Chicken Pasta', 'Alfredo Chicken Pasta', 'French Fries', 'Veg', 'Chicken', 'Fish Finger', 'Vegetable Omelette', 'Masala Omelette', 'Spanish Omelette', 'Cheese Omelette', 'Paneer Omelette', 'Ultimate Vegan Shakes', 'Bittersweet Symphony Shakes', 'The White Paradise Shakes', 'Deep Dark

Abyss Shakes', 'Irene Adlers Mystery Shakes', 'Red Weather Parody Shakes', 'Mind palace Shakes', 'Tower of Baskerville Shakes', 'Nights Demon Shakes', 'Black and White Romance Shakes', 'Day Dream Shakes', 'Sherlocks Addiction Shakes', 'Mrs Hudsons Secret Shakes', 'Classic Waffles', 'Caramel Waffles', 'Choco Treat Waffles', 'Brownie Waffles', 'Nutella Crunch Waffles', 'Snowflakes Waffles', 'Rocky Road Waffles', 'Lights Out Waffles']

9

['Students Veg Combo', 'Office Veg Combo', 'Students Non Veg Combo', 'Office Non Veg Combo', 'Mughlai Non Veg Combo', 'Aloo Paratha Combo', 'Poori Aloo Jeera Combo', 'Poori Aloo Tomato Combo', 'Paneer Paratha Combo', 'Students Veg Combo', 'Office Veg Combo', 'Students Non Veg Combo', 'Office Non Veg Combo', 'Mughlai Non Veg Combo', 'Shami Kebab Roll', 'Mughlai Biryani Combo [Veg]', 'Nawabi Thali Combo', 'The Lajawab Gravy Combo [Veg]', 'Mughlai Meal For Two [Veg]', 'The After Party Starters', 'Royal Feast For Four [Veg]', 'The Galawati Roll', 'Mughlai Biryani Combo [Non Veg]', 'The Nihari Combo [Chicken]', 'The Nihari Combo (Mutton)', 'The Lajawab Gravy Combo [Non Veg]', 'Mughlai Meal For Two [Non-Veg]', 'Biryani Blast', 'Royal Feast For Four [Non-Veg]', 'Baby Corn 65', 'Chilli Baby Corn', 'Mushroom 65', 'Chilli Mushroom', 'Veg Seekh Kebab', 'Kasturi Paneer Tikka', 'Paneer 65', 'Paneer Angara', 'Chilli Paneer', 'Hara Bhara Kebab', 'Paneer Tikka', 'Paneer Malai Tikka', 'Tandoori Veg Platter', 'Tandoori Chicken', 'Mutton Shami Kebab [Signature]', 'Mutton Galouti Kebab [Signature]', 'Chicken 65', 'Chicken Seekh Kebab', 'Chicken Tikka', 'Chilli Chicken', 'Chicken Angara', 'Chicken Kasturi Kebab', 'Chicken Malai Tikka', 'Mutton Sultani Seekh Kebab', 'Non Veg Tandoori Platter', 'Aloo Tomato Gravy', 'Aloo Capsicum Dry', 'Aloo Jeera Dry', 'Dal Fry', 'Dal Tadka', 'Rajma', 'Chole', 'Dal Makhani [Signature]', 'Lipta Mushroom Masala', 'Veg Kolhapuri', 'Chole Curry', 'Paneer Bhurji', 'Miloni Tarkari [Signature]', 'Mixed Veg', 'Palak Paneer', 'Diwani Handi', 'Mattar Paneer', 'Paneer Do Pyaza', 'Butter Paneer', 'Kadai Paneer [Signature]', 'Paneer Tikka Masala [Signature]', 'Methi Malai Paneer', 'Paneer Lababdar [Signature]', 'Murg Musallam', 'Egg Bhurji', 'Egg Curry', 'Chicken Nihari [Signature]', 'Chicken Curry', 'Chicken Do Pyaza', 'Chicken Methi Khas', 'Hyderabadi Chicken', 'Chicken Hara Pyaza', 'Chicken Kali Mirch', 'Chicken Kohlapuri', 'Kadai Chicken', 'Chicken Lababdar', 'Chicken Lajawab [Signature]', 'Chicken Tikka Masala', 'Chicken Boti Kebab Masala', 'Butter Chicken', 'Mughlai Chicken Masala [Signature]', 'Mutton Nihari [Signature]', 'Mutton Do Pyaza', 'Mutton Korma', 'Mutton Rogan Josh', 'Mughlai Mutton Masala', 'Mutton Boti Kebab Masala [Signature]', 'Murg Musallam', 'Steamed Rice', 'Jeera Rice', 'Ghee Rice', 'Khushka', 'Pulao', 'Veg Biryani', 'Mughlai Chicken Dum Biryani [Signature]', 'Chicken Tikka Biryani', 'Mughlai Mutton Dum Biryani [Signature]', 'Tandoori Roti', 'Lachha Paratha', 'Naan', 'Amritsari Kulcha', 'Garlic Naan', 'Missi Roti', 'Mughlai Paratha', 'Poori', 'Mughlai Bread Basket', 'Papad', 'Curd', 'Boondi Raita', 'Mixed Raita', 'Masala Papad', 'Pineapple Raita', 'Paneer Roll', 'Hawker Style Boiled Egg', 'Masala Omelette', 'Chicken Roll', 'Chicken Boti Roll', 'Shami Roll', 'The Galouti Roll [Signature]', 'Mutton Boti Roll', 'Gulab Jamun [2 Pieces]', 'Ice Cream Scoop [single]', 'Rice Kheer', 'Awadhi Kheer [Signature]', 'Gajar Ka Halwa', 'Lachhedar Rabri', 'Shahi Tukda [Signature]', 'Thums Up [750 Ml]', 'Mineral

Water [1 Litre]', 'Buttermilk', 'Lime Water', 'Thums Up (600 Ml)', 'Jal Jeera', 'Lassi', 'Lime Water Salt', 'Lime Water Sweet', 'Chocolate Lassi', 'Strawberry Lassi', 'Alphonso Mango Lassi', 'Thandai', 'Mango Lassi'] 9

...
['Veg Lasagna', 'Peri Peri Paneer', 'Chicken Steak', 'Schezwan Veg Noodles', 'Chicken Chilly Coriander', 'Plain French Fries', 'Crispy Corn', 'Nachos', 'Chicken Wings', 'Non Veg Alfredo Pasta', 'Farmhouse Pizza', 'Tomato Basil Soup', 'Veg Minestrone Soup', 'Cream of Mushroom Soup', 'Chicken Minestrone Soup', 'Cream of Chicken Soup', 'Veg Greek Salad', 'Veg Ceasar Salad', 'Veg Mexican Salad', 'Veg Russian Salad', 'Chicken Greek Salad', 'Chicken Ceasar Salad', 'Chicken Mexican Salad', 'Chicken Russian Salad', 'Gobi Manchurian', 'Gobi Chilly', 'Mushroom Chilly', 'Paneer 65', 'Paneer Chilly', 'Honey Chilly Potato', 'Egg Chilly', 'Egg Manchurian', 'Chicken Chilly Coriander', 'Fish Chilly Coriander', 'Chilly Fish', 'Plain French Fries', 'Masala French Fries', 'Crispy Corn', 'Bruschetta', 'Plain Garlic Bread', 'Cheese Garlic Bread', 'Pizza Sticks', 'Nachos', 'Chicken Wings', 'Fish Fingers', 'Veg Thai Green Curry', 'Veg Thai Red Curry', 'Veg Lasagna', 'Peri Peri Paneer', 'Mexican Cottage Chesse', 'Chicken Thai Green Curry', 'Chicken Thai Red Curry', 'Chicken Lasagna', 'Peri Peri Chicken', 'Chicken Steak', 'Grilled Fish in Lemon Butter Sauce', 'Chicken Stroganoff', 'Grilled Fish in Lemon Butter Sauce', 'Fish and Chips', 'Veg Alfredo Pasta', 'Veg Arrabiata Pasta', 'Veg Burnt Garlic Pasta', 'Veg Spaghetti Agli-e-Olio', 'Veg Basil Pesto Pasta', 'Non Veg Basil Pesto Pasta', 'Non Veg Spaghetti Agli-e-Olio', 'Non Veg Alfredo Pasta', 'Non Veg Arrabiata Pasta', 'Non Veg Burnt Garlic Pasta', 'Margherita Pizza', 'Farmhouse Pizza', 'Veg Basil Pesto Pizza', 'Chef's Inhouse Paneer Pizza', 'Chef's Inhouse Chicken Pizza', 'Non Veg Basil Pesto Pizza', 'Chicken Tikka Pizza', 'Veg Fried Rice', 'Schezwan Fried Rice', 'American Fried Rice', 'Chicken Fried Rice', 'Schezwan Chicken Fried Rice', 'American Chicken Fried Rice', 'Veg Noodles', 'American Noodles Veg', 'Schezwan Veg Noodles', 'Chicken Noodles', 'Schezwan Chicken Noodles', 'American Chicken Noodles', 'Veg Burger', 'Paneer Burger', 'Chicken Burger', 'Peri Peri Chicken Burger', 'Mixed Veg Wrap', 'Paneer Tikka Wrap', 'Egg and Cheese Wrap', 'Chicken Tikka Wrap']

1

['Konaseema Kodi Vepudu', 'Boneless Kodi Chips', 'Bangla Kodi', 'Kodi Roast', 'Jeedipappu Kodi Pakodi', 'Bamboo Chicken', 'Pachmirchi Kodi Kabab', 'Godavari Royyala Vepudu', 'Pappu Charu Annam', 'Gutti Vankaya Pulao', 'Ulavacharu Veg Pulao', 'Daddojanam', 'Raju Gari Kodi Pulao', 'Kodi Vepudu Pulao', 'Rayala Vari Kodi Pulao', 'Avakai Kodi Pulao', 'Boneless Pachi Mirchi Kodi Pulao', 'Gadwal Kodi Pulao', 'Royyala Vepudu Pulao', 'Chicken Dum Biryani', 'Mutton Dum Biryani', 'Ulavacharu', 'Gongura Kodi Kura', 'Veg Hot and Sour Soup', 'Veg Sweet Corn Soup', 'Veg Manchow Soup', 'Veg Lemon Coriander Soup', 'Non Veg Hot and Sour Soup', 'Non Veg Sweet Corn Soup', 'Non Veg Manchow Soup', 'Non Veg Lemon Coriander Soup', 'Garden Green Salad', 'Curd Onion Salad', 'Veg Seek Kabab', 'Hara Bhara Kabab', 'Paneer Tikka', 'Mokkajonna Vepudu', 'Veg Pandu Mirapakai', 'Chilly Paneer', 'Veg Crispy', 'Chilly Mushroom', 'Corn Kernel', 'Konaseema Kodi Vepudu', 'Boneless Kodi Chips', 'Bangla Kodi', 'Boneless Kotha Kodi', 'Kodi Roast', 'Jeedipappu Kodi Pakodi', 'Kodi Keema Balls', 'Bamboo Chicken',

'Boneless Peanut Chicken', 'Boneless Chinese Chilly Chicken', 'Boneless Schezwan Chicken', 'Boneless Crispy Chilly Chicken', 'Gogura Kodi Kabab', 'Pachmirchi Kodi Kabab', 'Pandu Mirapakaya Kodi Kabab', 'Murgh Malai Kabab', 'Boneless Mamsam Keema Balls', 'Mamsam Roasted', 'Boneless Konaseema Mamsam Vepudu', 'Boneless Mamsam Kaju Vepudu', 'Boneless Mamsam Miryala Vepudu', 'Boneless Korameenu Vepudu', 'Godavari Royyala Vepudu', 'Boneless Apollo Fish', 'Boneless Chilly Fish', 'Boneless Kolleti Chapala Vepudu', 'Loose Prawns', 'Chilly Prawns', 'Dal Fry', 'Dal Tadka', 'Veg Kandhari', 'Mixed Veg Curry', 'Methi Chaman', 'Paneer Butter Masala', 'Paneer Tikka Masala', 'Palak Paneer', 'Achari Paneer', 'Baby Corn Kaju Masala', 'Mushroom Tomato Curry', 'Chicken Masala', 'Boneless Chicken Chatpata', 'Boneless Butter Chicken', 'Punjabi Chicken', 'Boneless Methi Chicken', 'Ulavacharu', 'Lamb Rogan Josh', 'Andhra Kodi Kura', 'Miriyaala Kodi Kura', 'Avakai Kodi Kura', 'Gongura Kodi Kura', 'Telangana Kodi Kura', 'Mamsam Pappucharu', 'Telangana Mamsam Kura', 'Keema Kura', 'Gongura Mamsam', 'Royyala Iguru', 'Gongura Royyalu', 'Phulka', 'Tandoori Roti', 'Naan', 'Butter Naan', 'Garlic Naan', 'Plain Paratha', 'Stuffed Paratha', 'Masala Kulcha', 'Stuffed Egg Paratha', 'Stuffed Keema Paratha', 'Pappu Charu Annam', 'Gutti Vankaya Pulao', 'Avakai Veg Pulao', 'Paneer Tikka Pulao', 'Pachi Mirchi Veg Pulao', 'Ulavacharu Veg Pulao', 'Daddojanam', 'Curd Rice', 'Plain Rice', 'Egg White Pulao', 'Raju Gari Kodi Pulao', 'Kodi Vepudu Pulao', 'Rayala Vari Kodi Pulao', 'Avakai Kodi Pulao', 'Boneless Pachi Mirchi Kodi Pulao', 'Gadwal Kodi Pulao', 'Pachi MirchiKeema Pulao', 'Mamsam Vepudu Pulao', 'Keema Vepudu Pulao', 'Pachi Mirchi Royyala Pulao', 'Royyala Vepudu Pulao', 'Veg Biryani', 'Chicken Dum Biryani', 'Mutton Dum Biryani', 'Veg Fried Rice', 'Veg Schezwan Fried Rice', 'Paneer Mashed Rice', 'Egg Fried Rice', 'Chicken Fried Rice', 'Schezwan Chicken Fried Rice', 'Mixed Fried Rice', 'Veg Soft Noodles', 'Veg Schezwan Noodles', 'Chicken Noodles', 'Schezwan Chicken Noodles', 'Omelette', 'Curd', 'Apricot Delight', 'Buttermilk', 'No.1 Buttermilk', 'Mineral Water', 'Fresh Lime Water', 'Fresh Lime Soda', 'Lassi'] 1

['ST. Johns Chicken Parcel', 'BBQ Chicken', 'Ratatouille', 'Humptom Steak', 'Steak Diane', 'Oppure Steak', 'Beef Stroganoff', 'Fried Ice Cream', 'Caramel Custard', 'Peri Peri Fries', 'Ultimate Veg Nachos', 'Spicy Moroccan Egg [6 Pieces]', 'Ultimate Chicken Nachos', 'Steak Burger', 'Beef Burger', 'Chicken Burger', 'Arrabbiata Pasta', 'Alfredo Pasta', 'Mushroom Cappuccino Soup', 'Minesstrone Soup', 'Cock A Leekie Soup', 'Lamb Hungarian Soup', 'Peperonata Salad', 'Mediterranean Pasta Salad', 'Classic Caesar Salad', 'Roasted Beef Salad [York ST. Special]', 'York Special Tuna Salad', 'Chefs Special Salad', 'Ultimate Veg Nachos', 'Penang Crispy Paneer', 'Two In Punch [6 Pieces]', 'Crispy Onion Rings', 'Spicy Moroccan Egg [6 Pieces]', 'Ultimate Chicken Nachos', 'Chicken Fingers [6 Pieces]', 'American Corn Cheese Bolls [6 Pieces]', 'Grilled Harissa Prawns', 'Cajun Spiced Fish Fingers [8 Pieces]', 'Prawns Olive Skewers [3 Skewers]', 'Panko Crusted Chicken [6 Pieces]', 'Deep Fried Prawns [8 Pieces]', 'Fisherman's Choice', 'Vegetable Shashuk Forage', 'Vegetable Moussaka', 'Ratatouille', 'Boston Bake', 'Vegetable Steak', 'Vegetable Stir Fry', 'Mexican Green Rice with Tortilla Chips and Salsa', 'Stuffed Capsicum and Tomato Morray', 'Humptom Steak', 'Steak Diane', 'Oppure Steak', 'Beef Stroganoff', 'Arrabbiata Pasta', 'Alfredo Pasta', 'Basilica Pasta', 'Paprika Pasta', 'Marinara Pasta',

'Bolognasie Sauce', 'Veg Burger', 'Roasted Vegetable Burger', 'Steak Burger', 'Beef Burger', 'Chicken Burger', 'Lamb Burger', 'Roasted Vegetable Sandwich', 'Chicken Grilled Sandwich', 'BBQ Chicken Sandwich', 'Beef Grilled Sandwich', 'Tuna Grilled Sandwich', 'Garlic Toast', 'Fries', 'Peri Peri Fries', 'Cheese Fries', 'Cheese Toast', 'Chilli Cheese Toast', 'Wiener Schnitzel', 'ST. Johns Chicken Parcel', 'Triple Decker Chicken', 'Black Pepper Chicken', 'BBQ Chicken', 'Oven Roasted Herb Chicken', 'Lemon Basil Chicken Roller', 'Herbs Crusted Grilled Fish', 'Fish N Chips with Peas', 'Fisherman's Bake', 'Grilled Fish with Lemon Butter Sauce', 'Fried Ice Cream', 'Caramel Custard', 'Proline Suffle', 'Panna Cotta', 'Sizzling Brownie', 'Green Apple Mojito', 'Orange Mojito', 'Blue Curacao Mojito', 'Strawberry Mojito', 'Passion Fruits Mojito', 'Peach Iced Tea', 'Lemon Iced Tea', 'Espresso Single Coffee', 'Espresso Double Coffee', 'Americano Coffee', 'Cappuccino Coffee', 'Cafe Latte Coffee', 'Cafe Mocha Coffee', 'Hot Chocolate Coffee', 'Iced Latte Cold Coffee', 'Mocha Iceberg Cold Coffee', 'Frozen Milano Cold Coffee', 'Brownie Shake Cold Coffee', 'Ferrero Rocher Shake Cold Coffee', 'Frozen Mango Frappe', 'Frozen Strawberry Frappe', 'Litchi Frappe', 'Kiwi Frappe', 'Mango Kiwi Smoothie', 'Ultimate Berry Shake', 'Basil Spitzer']

1

['Crispy Paneer Burger', 'Veg Burger', 'Egg Burger', 'Chicken Burger', 'Crispy Chicken Burger', 'Mutton Burger', 'Orange Juice', 'Oreo Milkshake', 'Chicken Burger Combo', 'Talayani Subwich', 'Milano Subwich', 'Rambo Subwich', 'Secret Garden Subwich', 'Talayani Subwich', 'Milano Subwich', 'Rambo Subwich', 'Veg Burger Combo', 'Crispy Paneer Burger Combo', 'Chicken Burger Combo', 'Mutton Burger Combo', 'Crispy Paneer Burger', 'Veg Burger', 'Egg Burger', 'Chicken Burger', 'Crispy Chicken Burger', 'Mutton Burger', 'Plain Omelette', 'Egg White Omelette', 'Masala Omelette', 'Chicken Omelette', 'Chicken Ham Omelette', 'Vanilla Ice Cream', 'Butterscotch Ice Cream', 'Strawberry Ice Cream', 'Mango Ice Cream', 'Chocolate Ice Cream', 'Vanilla and Butterscotch Sundae', 'Dry fruit Dark chocolate Ice Cream', 'Strawberry and Mango Delight', 'Orange Juice', 'Mosambi Juice', 'Watermelon Juice', 'Muskmelon Juice', 'Beetroot Juice', 'Pineapple Juice', 'Grape Juice', 'Fresh Lime', 'Fresh Lime Soda', 'Mango Milkshake', 'Pineapple Milkshake', 'Strawberry Milkshake', 'Vanilla Milkshake', 'Butterscotch Milkshake', 'Cold Chocolate Milkshake', 'Cold Coffee Milkshake', 'Oreo Milkshake', 'Pomegranate Milkshake', 'Orange Pure Juice', 'Mosambi Pure Juice', 'Pomegranate Pure Juice', 'Pineapple Pure Juice', 'Grape Pure Juice', 'Watermelon Pure Juice']

1

['Veg Mudde Combo', 'Veg Mini Meal Combo', 'Veg Nammura Oota Combo', 'Non Veg Mudde Oota Combo', 'Chicken Biryani Combo', 'Naati Manae Paya Soup', 'Naati Koli Fry', 'Puliyogare', 'Chicken Donne Biryani', 'Naati Koli Biryani', 'Mutton Donne Biryani', 'Naati Manae Soppina Saru', 'Naati Koli Saru', 'Veg Dosa Combo', 'Veg Chapati Combo', 'Veg Coin Paratha Combo', 'Veg Chitranna Combo', 'Veg Puliyogare Combo', 'Veggie Snack Combo', 'Veg Mudde Combo', 'Veg Mini Meal Combo', 'Veg Nammura Oota Combo', 'Veg Super Combo', 'Non Veg Dosa Combo', 'Non Veg Chapati Combo', 'Non Veg Coin Paratha Combo', 'Non Veg Mudde Oota Combo', 'Non Veg Nammura Oota Combo', 'Chicken Biryani Combo', 'Handi Biryani Combo', 'Chicken

Starter Combo', 'Nammura Mutton Combo', 'Mutton Starter Combo', 'Mangalore Fish Fry and Fish Curry Rice Combo', 'Mangalore Fish Curry Rice Mini Combo', 'Naati Manae Paya Soup', 'Assorted Bajjis', 'Gobi Manchurian', 'Gobi Majestic Fry', 'Gobi 65', 'Naati Style Onion Pakoda', 'Onion Rings', 'Chilli Bajji', 'Potato Bajji', 'Capsicum Bajji', 'Naati Koli Fry', 'Hasi Menasina Koli Fry', 'Kempu Menasina Koli Fry', 'Chicken Kebab', 'Chicken Lollipop', 'Boneless Chicken Sholay Kebab', 'Naati Manae Special Chicken Platter', 'Nammura Mutton Fry', 'Mutton Liver Fry', 'Mutton Boti Fry', 'Mutter Pepper Fry', 'Naati Manae Soppina Saru', 'Naati Manae Sambar', 'Naati Manae Traditional Rasam', 'Naati Manae Veg Kurma', 'Chicken Chops Gravy', 'Naati Koli Saru', 'Mutton Chops Gravy', 'Mangalore Fish Curry', 'Chapati', 'Coin Paratha', 'Chitranna', 'Puliyogare', 'Jeera Rice', 'Mosuranna', 'Veg Dum Biryani', 'Egg Biryani', 'Chicken Donne Biryani', 'Naati Koli Biryani', 'Chicken Hasi Menasina Donne Biryani', 'Kempu Menasina Donne Biryani', 'Boneless Chicken Handi Biryani', 'Chicken Handi Biryani', 'Kebab and Biryani', 'Lollipop Biryani', 'Biryani Rice', 'Mutton Donne Biryani', 'Naati Manae Special Dosa', 'Raagi Ball', 'Karnataka Traditional Buttermilk']

1

Name: menu_item, Length: 9098, dtype: int64

[354]: df1.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51717 entries, 0 to 51716
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   url                   51717 non-null  object
 1   address              51717 non-null  object
 2   name                 51717 non-null  object
 3   online_order         51717 non-null  object
 4   book_table           51717 non-null  object
 5   rate                 43942 non-null  object
 6   votes                51717 non-null  int64
 7   phone                50509 non-null  object
 8   location              51696 non-null  object
 9   rest_type            51490 non-null  object
10  dish_liked           23639 non-null  object
11  cuisines              51672 non-null  object
12  approx_cost(for two people) 51371 non-null  object
13  reviews_list         51717 non-null  object
14  menu_item             51717 non-null  object
15  listed_in(type)       51717 non-null  object
16  listed_in(city)       51717 non-null  object
dtypes: int64(1), object(16)
memory usage: 6.7+ MB

```



```
[355]: df1.head()
```

```
[355]:                                     url \
0 https://www.zomato.com/bangalore/jalsa-banasha...
1 https://www.zomato.com/bangalore/spice-elephan...
2 https://www.zomato.com/SanchurroBangalore?cont...
3 https://www.zomato.com/bangalore/addhuri-udupi...
4 https://www.zomato.com/bangalore/grand-village...

                                     address                                     name \
0 942, 21st Main Road, 2nd Stage, Banashankari, ... Jalsa
1 2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ... Spice Elephant
2 1112, Next to KIMS Medical College, 17th Cross... San Churro Cafe
3 1st Floor, Annakuteera, 3rd Stage, Banashankar... Addhuri Udupi Bhojana
4 10, 3rd Floor, Lakshmi Associates, Gandhi Baza... Grand Village

online_order book_table  rate  votes  phone \
0 Yes Yes 4.1/5 775 080 42297555\r\n+91 9743772233
1 Yes No 4.1/5 787 080 41714161
2 Yes No 3.8/5 918 +91 9663487993
3 No No 3.7/5 88 +91 9620009302
4 No No 3.8/5 166 +91 8026612447\r\n+91 9901210005

location rest_type \
0 Banashankari Casual Dining
1 Banashankari Casual Dining
2 Banashankari Cafe, Casual Dining
3 Banashankari Quick Bites
4 Basavanagudi Casual Dining

dish_liked \
0 Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1 Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2 Churros, Cannelloni, Minestrone Soup, Hot Choc...
3 Masala Dosa
4 Panipuri, Gol Gappe

cuisines approx_cost(for two people) \
0 North Indian, Mughlai, Chinese 800
1 Chinese, North Indian, Thai 800
2 Cafe, Mexican, Italian 800
3 South Indian, North Indian 300
4 North Indian, Rajasthani 600

reviews_list menu_item \
0 [('Rated 4.0', 'RATED\n A beautiful place to ... []
1 [('Rated 4.0', 'RATED\n Had been here for din... []
```

```

2  [('Rated 3.0', 'RATED\n  Ambience is not that ...      []
3  [('Rated 4.0', 'RATED\n  Great food and proper...      []
4  [('Rated 4.0', 'RATED\n  Very good restaurant ...      []

```

```

    listed_in(type) listed_in(city)
0      Buffet      Banashankari
1      Buffet      Banashankari
2      Buffet      Banashankari
3      Buffet      Banashankari
4      Buffet      Banashankari

```

```
[356]: df1['listed_in(type)'].value_counts()
```

```

[356]: Delivery      25942
Dine-out      17779
Desserts      3593
Cafes      1723
Drinks & nightlife      1101
Buffet      882
Pubs and bars      697
Name: listed_in(type), dtype: int64

```

0.0.1 Dropping few columns:

```
[357]: df1.columns
```

```

[357]: Index(['url', 'address', 'name', 'online_order', 'book_table', 'rate', 'votes',
            'phone', 'location', 'rest_type', 'dish_liked', 'cuisines',
            'approx_cost(for two people)', 'reviews_list', 'menu_item',
            'listed_in(type)', 'listed_in(city)'],
            dtype='object')

```

```

[358]: df = df1.
        ↪drop(['address', 'phone', 'dish_liked', 'cuisines', 'reviews_list', 'listed_in(city)', 'reviews_l
        ↪axis = 1)

```

```
[359]: df.head()
```

```

[359]:
           url                                     name \
0  https://www.zomato.com/bangalore/jalsa-banasha...      Jalsa
1  https://www.zomato.com/bangalore/spice-elephan...  Spice Elephant
2  https://www.zomato.com/SanchurroBangalore?cont...  San Churro Cafe
3  https://www.zomato.com/bangalore/addhuri-udupi...  Addhuri Udupi Bhojana
4  https://www.zomato.com/bangalore/grand-village...  Grand Village

  online_order  book_table  rate  votes  location  rest_type \
0           Yes          Yes  4.1/5    775  Banashankari  Casual Dining

```

1	Yes	No	4.1/5	787	Banashankari	Casual Dining
2	Yes	No	3.8/5	918	Banashankari	Cafe, Casual Dining
3	No	No	3.7/5	88	Banashankari	Quick Bites
4	No	No	3.8/5	166	Basavanagudi	Casual Dining

	approx_cost(for two people)	listed_in(type)
0	800	Buffet
1	800	Buffet
2	800	Buffet
3	300	Buffet
4	600	Buffet

```
[360]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51717 entries, 0 to 51716
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   url                                    51717 non-null  object
1   name                                    51717 non-null  object
2   online_order                          51717 non-null  object
3   book_table                            51717 non-null  object
4   rate                                    43942 non-null  object
5   votes                                  51717 non-null  int64
6   location                              51696 non-null  object
7   rest_type                             51490 non-null  object
8   approx_cost(for two people)           51371 non-null  object
9   listed_in(type)                       51717 non-null  object
dtypes: int64(1), object(9)
memory usage: 3.9+ MB
```

- Observations :
- Rating is in string,
- approx cost is also in string

```
[361]: df['rate'].unique()
```

```
[361]: array(['4.1/5', '3.8/5', '3.7/5', '3.6/5', '4.6/5', '4.0/5', '4.2/5',
        '3.9/5', '3.1/5', '3.0/5', '3.2/5', '3.3/5', '2.8/5', '4.4/5',
        '4.3/5', 'NEW', '2.9/5', '3.5/5', nan, '2.6/5', '3.8 /5', '3.4/5',
        '4.5/5', '2.5/5', '2.7/5', '4.7/5', '2.4/5', '2.2/5', '2.3/5',
        '3.4 /5', '-', '3.6 /5', '4.8/5', '3.9 /5', '4.2 /5', '4.0 /5',
        '4.1 /5', '3.7 /5', '3.1 /5', '2.9 /5', '3.3 /5', '2.8 /5',
        '3.5 /5', '2.7 /5', '2.5 /5', '3.2 /5', '2.6 /5', '4.5 /5',
        '4.3 /5', '4.4 /5', '4.9/5', '2.1/5', '2.0/5', '1.8/5', '4.6 /5',
        '4.9 /5', '3.0 /5', '4.8 /5', '2.3 /5', '4.7 /5', '2.4 /5',
        '2.1 /5', '2.2 /5', '2.0 /5', '1.8 /5'], dtype=object)
```

```
[362]: # there is NEW, - lets take care of this
# Note in pandas if there is missing data it will shows as nan and it will
↳ ignore that from computation, dont show any error. . skip NaNs by default
def cleaning_ratings(rating):
    if(rating == 'NEW' or rating == '-'):
        return np.nan
    else:
        return float(str(rating).split('/')[0])
```

```
[363]: df['rate']
```

```
[363]: 0      4.1/5
1      4.1/5
2      3.8/5
3      3.7/5
4      3.8/5
...
51712  3.6 /5
51713    NaN
51714    NaN
51715  4.3 /5
51716  3.4 /5
Name: rate, Length: 51717, dtype: object
```

```
[364]: df['rate'].apply(cleaning_ratings)
```

```
[364]: 0      4.1
1      4.1
2      3.8
3      3.7
4      3.8
...
51712  3.6
51713    NaN
51714    NaN
51715  4.3
51716  3.4
Name: rate, Length: 51717, dtype: float64
```

```
[365]: #Redefine rate
df['rate'] = df['rate'].apply(cleaning_ratings)
df.head()
```

```
[365]:
```

	url	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	San Churro Cafe

```

3 https://www.zomato.com/bangalore/addhuri-udupi... Addhuri Udupi Bhojana
4 https://www.zomato.com/bangalore/grand-village... Grand Village

```

	online_order	book_table	rate	votes	location	rest_type \
0	Yes	Yes	4.1	775	Banashankari	Casual Dining
1	Yes	No	4.1	787	Banashankari	Casual Dining
2	Yes	No	3.8	918	Banashankari	Cafe, Casual Dining
3	No	No	3.7	88	Banashankari	Quick Bites
4	No	No	3.8	166	Basavanagudi	Casual Dining

	approx_cost(for two people)	listed_in(type)
0	800	Buffet
1	800	Buffet
2	800	Buffet
3	300	Buffet
4	600	Buffet

```

[366]: # Checking how many null values (missing values) are there:
df.isnull().sum()

```

```

[366]: url                0
      name                0
      online_order        0
      book_table          0
      rate              10052
      votes              0
      location           21
      rest_type          227
      approx_cost(for two people)  346
      listed_in(type)      0
      dtype: int64

```

```

[367]: np.round(df['rate'].mean(),1)

```

```

[367]: 3.7

```

0.0.2 Filling the missing data in rating with mean ratings:

```

[368]: df['rate'].fillna(np.round(df['rate'].mean(),1))

```

```

[368]: 0      4.1
      1      4.1
      2      3.8
      3      3.7
      4      3.8
      ...
      51712  3.6

```

```

51713    3.7
51714    3.7
51715    4.3
51716    3.4
Name: rate, Length: 51717, dtype: float64

```

```
[369]: df['rate'] = df['rate'].fillna(np.round(df['rate'].mean(),1))
df['rate'].isnull().sum()
```

```
[369]: 0
```

```
[370]: df.head()
```

```
[370]:
```

	url	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	San Churro Cafe
3	https://www.zomato.com/bangalore/addhuri-udupi...	Addhuri Udupi Bhojana
4	https://www.zomato.com/bangalore/grand-village...	Grand Village

	online_order	book_table	rate	votes	location	rest_type \
0	Yes	Yes	4.1	775	Banashankari	Casual Dining
1	Yes	No	4.1	787	Banashankari	Casual Dining
2	Yes	No	3.8	918	Banashankari	Cafe, Casual Dining
3	No	No	3.7	88	Banashankari	Quick Bites
4	No	No	3.8	166	Basavanagudi	Casual Dining

	approx_cost(for two people)	listed_in(type)
0	800	Buffet
1	800	Buffet
2	800	Buffet
3	300	Buffet
4	600	Buffet

```
[371]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51717 entries, 0 to 51716
Data columns (total 10 columns):
#   Column              Non-Null Count  Dtype
---  -
0   url                  51717 non-null  object
1   name                 51717 non-null  object
2   online_order         51717 non-null  object
3   book_table          51717 non-null  object
4   rate                 51717 non-null  float64
5   votes                51717 non-null  int64

```

```

6    location                51696 non-null  object
7    rest_type               51490 non-null  object
8    approx_cost(for two people) 51371 non-null object
9    listed_in(type)         51717 non-null  object
dtypes: float64(1), int64(1), object(8)
memory usage: 3.9+ MB

```

```
[372]: # approx_cost(for two people) 51371 non-null object : we need to fix this
```

```
[373]: df['approx_cost(for two people)'].unique()
```

```
[373]: array(['800', '300', '600', '700', '550', '500', '450', '650', '400',
        '900', '200', '750', '150', '850', '100', '1,200', '350', '250',
        '950', '1,000', '1,500', '1,300', '199', '80', '1,100', '160',
        '1,600', '230', '130', '50', '190', '1,700', nan, '1,400', '180',
        '1,350', '2,200', '2,000', '1,800', '1,900', '330', '2,500',
        '2,100', '3,000', '2,800', '3,400', '40', '1,250', '3,500',
        '4,000', '2,400', '2,600', '120', '1,450', '469', '70', '3,200',
        '60', '560', '240', '360', '6,000', '1,050', '2,300', '4,100',
        '5,000', '3,700', '1,650', '2,700', '4,500', '140'], dtype=object)
```

```
[374]: # int('1,200'.replace(',','')) This is the logic we do, since we can convert
        ↳ directly to float/int
```

```
[375]: #df['approx_cost(for two people)'].isnull().sum()
df.isnull().sum()
```

```
[375]: url                0
name                    0
online_order           0
book_table             0
rate                  0
votes                 0
location              21
rest_type             227
approx_cost(for two people) 346
listed_in(type)        0
dtype: int64
```

- 346 null values :
- And in total approx ~600 rown at max in total, If i drop all those rows it wont affect, because my data set length is alose to 50k.

```
[376]: # Hence we can remove thows rows:
df.dropna()
```

[376]:

	url \				
0	https://www.zomato.com/bangalore/jalsa-banasha...				
1	https://www.zomato.com/bangalore/spice-elephan...				
2	https://www.zomato.com/SanchurroBangalore?cont...				
3	https://www.zomato.com/bangalore/addhuri-udupi...				
4	https://www.zomato.com/bangalore/grand-village...				
...	...				
51712	https://www.zomato.com/bangalore/best-brews-fo...				
51713	https://www.zomato.com/bangalore/vinod-bar-and...				
51714	https://www.zomato.com/bangalore/plunge-sherat...				
51715	https://www.zomato.com/bangalore/chime-sherato...				
51716	https://www.zomato.com/bangalore/the-nest-the-...				
		name	online_order	\	
0		Jalsa	Yes		
1		Spice Elephant	Yes		
2		San Churro Cafe	Yes		
3		Addhuri Udupi Bhojana	No		
4		Grand Village	No		
...	...	...	...		
51712	Best Brews - Four Points by Sheraton Bengaluru...		No		
51713	Vinod Bar And Restaurant		No		
51714	Plunge - Sheraton Grand Bengaluru Whitefield H...		No		
51715	Chime - Sheraton Grand Bengaluru Whitefield Ho...		No		
51716	The Nest - The Den Bengaluru		No		
	book_table	rate	votes	location \	
0	Yes	4.1	775	Banashankari	
1	No	4.1	787	Banashankari	
2	No	3.8	918	Banashankari	
3	No	3.7	88	Banashankari	
4	No	3.8	166	Basavanagudi	
...	...	...	...	...	
51712	No	3.6	27	Whitefield	
51713	No	3.7	0	Whitefield	
51714	No	3.7	0	Whitefield	
51715	Yes	4.3	236	ITPL Main Road, Whitefield	
51716	No	3.4	13	ITPL Main Road, Whitefield	
	rest_type	approx_cost(for two people)	listed_in(type)		
0	Casual Dining	800	Buffet		
1	Casual Dining	800	Buffet		
2	Cafe, Casual Dining	800	Buffet		
3	Quick Bites	300	Buffet		
4	Casual Dining	600	Buffet		
...	...	...	...		
51712	Bar	1,500	Pubs and bars		

51713	Bar	600	Pubs and bars
51714	Bar	2,000	Pubs and bars
51715	Bar	2,500	Pubs and bars
51716	Bar, Casual Dining	1,500	Pubs and bars

[51167 rows x 10 columns]

```
[377]: # Assigning it to df
df = df.dropna()
```

- We are losing 1% of data

```
[378]: df.isnull().sum()
```

```
[378]: url          0
name            0
online_order    0
book_table      0
rate            0
votes           0
location        0
rest_type       0
approx_cost(for two people)  0
listed_in(type)  0
dtype: int64
```

```
[379]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 51167 entries, 0 to 51716
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   url                                  51167 non-null  object
1   name                                51167 non-null  object
2   online_order                        51167 non-null  object
3   book_table                          51167 non-null  object
4   rate                                51167 non-null  float64
5   votes                               51167 non-null  int64
6   location                            51167 non-null  object
7   rest_type                           51167 non-null  object
8   approx_cost(for two people)         51167 non-null  object
9   listed_in(type)                     51167 non-null  object
dtypes: float64(1), int64(1), object(8)
memory usage: 4.3+ MB
```

- Now there is no null values in df

```
[380]: def cleaned_approx_cost(value):
        value = str(value)
        if ',' in value:
            return float(value.replace(',', ''))
        else:
            return float(value)
```

```
[381]: df['approx_cost(for two people)'].apply(cleaned_approx_cost)
```

```
[381]: 0      800.0
      1      800.0
      2      800.0
      3      300.0
      4      600.0
      ...
     51712    1500.0
     51713      600.0
     51714    2000.0
     51715    2500.0
     51716    1500.0
      Name: approx_cost(for two people), Length: 51167, dtype: float64
```

```
[382]: # reassign it to df
df['approx_cost(for two people)'] = df['approx_cost(for two people)'].
    ↪ apply(cleaned_approx_cost)
```

```
[383]: # Checking for duplicated rows
df.duplicated().sum()
```

```
[383]: 0
```

```
[384]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 51167 entries, 0 to 51716
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   url                   51167 non-null  object
 1   name                  51167 non-null  object
 2   online_order          51167 non-null  object
 3   book_table            51167 non-null  object
 4   rate                  51167 non-null  float64
 5   votes                 51167 non-null  int64
 6   location              51167 non-null  object
 7   rest_type             51167 non-null  object
```

```

8    approx_cost(for two people)  51167 non-null  float64
9    listed_in(type)              51167 non-null  object
dtypes: float64(2), int64(1), object(7)
memory usage: 4.3+ MB

```

0.0.3 Understand this rest_type column

```
[385]: df['rest_type'].value_counts()
```

```

[385]: Quick Bites                19048
       Casual Dining             10275
       Cafe                     3687
       Delivery                  2587
       Dessert Parlor            2245
       ...
       Dessert Parlor, Kiosk      2
       Dessert Parlor, Food Court 2
       Pop Up                    2
       Sweet Shop, Dessert Parlor 1
       Quick Bites, Kiosk         1
Name: rest_type, Length: 93, dtype: int64

```

```

[386]: rest_type = df['rest_type'].value_counts()
       rest_type
#rest_type[rest_type < 450]

```

```

[386]: Quick Bites                19048
       Casual Dining             10275
       Cafe                     3687
       Delivery                  2587
       Dessert Parlor            2245
       ...
       Dessert Parlor, Kiosk      2
       Dessert Parlor, Food Court 2
       Pop Up                    2
       Sweet Shop, Dessert Parlor 1
       Quick Bites, Kiosk         1
Name: rest_type, Length: 93, dtype: int64

```

```
[387]: rest_type[rest_type < 450]
```

```

[387]: Bar, Casual Dining        415
       Lounge                   396
       Pub                      357
       Fine Dining               346
       Casual Dining, Cafe       311
       ...

```

```
Dessert Parlor, Kiosk      2
Dessert Parlor, Food Court 2
Pop Up                    2
Sweet Shop, Dessert Parlor 1
Quick Bites, Kiosk        1
Name: rest_type, Length: 81, dtype: int64
```

```
[388]: type_below_500 = rest_type[rest_type < 500]
```

```
[389]: def clean_rest_type(value):
        if (value) in type_below_500:
            return 'Others'
        else:
            return value
```

```
[390]: df['rest_type'].value_counts()
```

```
[390]: Quick Bites      19048
        Casual Dining   10275
        Cafe            3687
        Delivery        2587
        Dessert Parlor   2245
        ...
        Dessert Parlor, Kiosk      2
        Dessert Parlor, Food Court 2
        Pop Up                    2
        Sweet Shop, Dessert Parlor 1
        Quick Bites, Kiosk        1
Name: rest_type, Length: 93, dtype: int64
```

```
[391]: clean_rest_type('Quick Bites')
```

```
[391]: 'Quick Bites'
```

```
[392]: clean_rest_type('Quick Bites, Kiosk')
```

```
[392]: 'Others'
```

```
[393]: df['rest_type'].apply(clean_rest_type)
```

```
[393]: 0      Casual Dining
        1      Casual Dining
        2      Others
        3      Quick Bites
        4      Casual Dining
        ...
51712      Bar
```

```

51713          Bar
51714          Bar
51715          Bar
51716      Others
Name: rest_type, Length: 51167, dtype: object

```

```

[394]: # Reassign
df['rest_type'] = df['rest_type'].apply(clean_rest_type)

```

```

[395]: df['rest_type'].value_counts()

```

```

[395]: Quick Bites          19048
Casual Dining            10275
Others                   6860
Cafe                     3687
Delivery                 2587
Dessert Parlor           2245
Takeaway, Delivery       2016
Bakery                   1141
Casual Dining, Bar       1136
Beverage Shop            867
Bar                      686
Food Court               619
Name: rest_type, dtype: int64

```

```

[396]: df['location'].value_counts()
#len(df['location'].value_counts())

```

```

[396]: BTM                  5071
HSR                      2496
Koramangala 5th Block    2481
JP Nagar                 2219
Whitefield               2117
...
West Bangalore           6
Yelahanka                5
Jakkur                   3
Rajarajeshwari Nagar     2
Peenya                   1
Name: location, Length: 93, dtype: int64

```

```

[397]: rest_loc = df['location'].value_counts()
rest_loc[rest_loc < 500]

```

```

[397]: Shivajinagar          499
Cunningham Road            491
Domlur                     482

```

Old Airport Road	437
Ejipura	434
Commercial Street	370
St. Marks Road	343
Koramangala 8th Block	294
Vasanth Nagar	293
Jeevan Bhima Nagar	268
Wilson Garden	246
Bommanahalli	236
Koramangala 3rd Block	216
Kumaraswamy Layout	194
Thippasandra	191
Basaveshwara Nagar	187
Nagawara	187
Seshadripuram	165
Hennur	159
Majestic	155
HBR Layout	153
Infantry Road	151
Race Course Road	139
City Market	122
Yeshwantpur	119
ITPL Main Road, Whitefield	113
Varthur Main Road, Whitefield	109
South Bangalore	107
Koramangala 2nd Block	102
Hosur Road	102
Kaggadasapura	101
CV Raman Nagar	90
RT Nagar	78
Vijay Nagar	78
Sanjay Nagar	76
Sadashiv Nagar	63
Sahakara Nagar	53
Koramangala	48
East Bangalore	43
Jalahalli	38
Magadi Road	34
Rammurthy Nagar	32
Sankey Road	27
Langford Town	27
Mysore Road	22
Old Madras Road	22
Kanakapura Road	19
KR Puram	18
Uttarahalli	17
Hebbal	14

North Bangalore	14
Nagarbhavi	9
Kengeri	9
Central Bangalore	8
West Bangalore	6
Yelahanka	5
Jakkur	3
Rajarajeshwari Nagar	2
Peenya	1

Name: location, dtype: int64

```
[398]: loc_below_500 = rest_loc[rest_loc < 500]
```

```
[399]: def clean_rest_loc(loc):
        if loc in loc_below_500:
            return 'Others'
        else:
            return loc
```

```
[400]: df['location'].apply(clean_rest_loc)
```

```
[400]: 0      Banashankari
        1      Banashankari
        2      Banashankari
        3      Banashankari
        4      Basavanagudi
        ...
        51712    Whitefield
        51713    Whitefield
        51714    Whitefield
        51715      Others
        51716      Others
Name: location, Length: 51167, dtype: object
```

```
[401]: df['location'] = df['location'].apply(clean_rest_loc)
        df['location'].value_counts()
```

```
[401]: Others      8021
        BTM        5071
        HSR        2496
        Koramangala 5th Block  2481
        JP Nagar    2219
        Whitefield  2117
        Indiranagar  2033
        Jayanagar   1916
        Marathahalli 1811
        Bannerghatta Road 1611
```

Bellandur	1271
Electronic City	1249
Koramangala 1st Block	1237
Brigade Road	1218
Koramangala 7th Block	1176
Koramangala 6th Block	1129
Sarjapur Road	1049
Koramangala 4th Block	1017
Ulsoor	1017
Banashankari	906
MG Road	894
Kalyan Nagar	843
Richmond Road	804
Malleshwaram	724
Frazer Town	720
Basavanagudi	684
Residency Road	674
Brookefield	656
Banaswadi	645
New BEL Road	644
Kammanahalli	640
Rajajinagar	591
Church Street	569
Lavelle Road	523
Shanti Nagar	511

Name: location, dtype: int64

```
[402]: df.head()
```

```
[402]:
```

	url	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	San Churro Cafe
3	https://www.zomato.com/bangalore/addhuri-udupi...	Addhuri Udupi Bhojana
4	https://www.zomato.com/bangalore/grand-village...	Grand Village

	online_order	book_table	rate	votes	location	rest_type \
0	Yes	Yes	4.1	775	Banashankari	Casual Dining
1	Yes	No	4.1	787	Banashankari	Casual Dining
2	Yes	No	3.8	918	Banashankari	Others
3	No	No	3.7	88	Banashankari	Quick Bites
4	No	No	3.8	166	Basavanagudi	Casual Dining

	approx_cost(for two people)	listed_in(type)
0	800.0	Buffet
1	800.0	Buffet
2	800.0	Buffet

3	300.0	Buffet
4	600.0	Buffet

```
[403]: df['listed_in(type)'].value_counts()
```

```
[403]: Delivery          25669
Dine-out              17586
Desserts              3559
Cafes                 1703
Drinks & nightlife    1091
Buffet                871
Pubs and bars         688
Name: listed_in(type), dtype: int64
```

0.0.4 Number of Resturants delivers online and offline order as per Zomato dataset:

```
[404]: df['online_order']
```

```
[404]: 0      Yes
1      Yes
2      Yes
3      No
4      No
...
51712   No
51713   No
51714   No
51715   No
51716   No
Name: online_order, Length: 51167, dtype: object
```

```
[405]: df['online_order'].value_counts()
```

```
[405]: Yes      30327
No       20840
Name: online_order, dtype: int64
```

```
[406]: print(f"There are {df['online_order'].value_counts().iloc[0]} number of_
↳resturants delivers order online in bangalore as per this Zomato data set.")
```

There are 30327 number of resturants delivers order online in bangalore as per this Zomato data set.

```
[407]: print(f"There are {df['online_order'].value_counts().iloc[1]} number of_
↳resturants delivers order offline in bangalore as per this Zomato data set.")
```

There are 20840 number of restaurants delivers order offline in bangalore as per this Zomato data set.

[]:

```
[408]: # Count the values
order_counts = df['online_order'].value_counts()
# plot using pandas library
# Plot with labels
order_counts.plot(
    kind='pie',
    labels=order_counts.index,    # These are your labels (e.g., 'Yes', 'No')
    autopct='%1.1f%%',          # Show percentage with 1 decimal
    ylabel='',                   # Remove y-label
    title='Online Order Distribution'
)
```

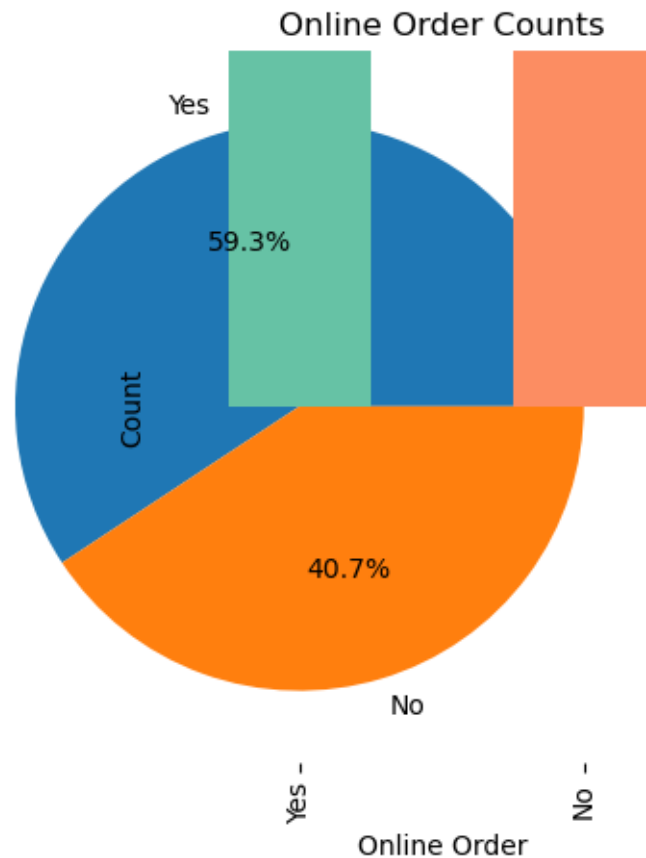
```
[408]: <Axes: title={'center': 'Online Order Distribution'}>
```

```
[409]: df['online_order'].value_counts().plot(kind = 'bar', color = sns.
        color_palette())
```

```
[409]: <Axes: title={'center': 'Online Order Distribution'}>
```

```
[410]: # Get a palette of colors
colors = sns.color_palette("Set2") # or "pastel", "muted", etc.

# Bar plot with Seaborn colors
df['online_order'].value_counts().plot(kind='bar', color=colors)
plt.title('Online Order Counts')
plt.ylabel('Count')
plt.xlabel('Online Order')
plt.show()
```



```
[411]: import matplotlib.pyplot as plt
import seaborn as sns

# Set journal-style aesthetics
sns.set_style("whitegrid")
plt.rcParams.update({
    'font.family': 'serif',
    'font.serif': ['DejaVu Serif'],
    'font.size': 14,
    'axes.labelsize': 16,
    'axes.titlesize': 18,
    'xtick.labelsize': 12,
    'ytick.labelsize': 12,
    'legend.fontsize': 12,
    'figure.dpi': 300
})

# Set figure size here
plt.figure(figsize=(5, 3)) # width=5 inches, height=3 inches
```

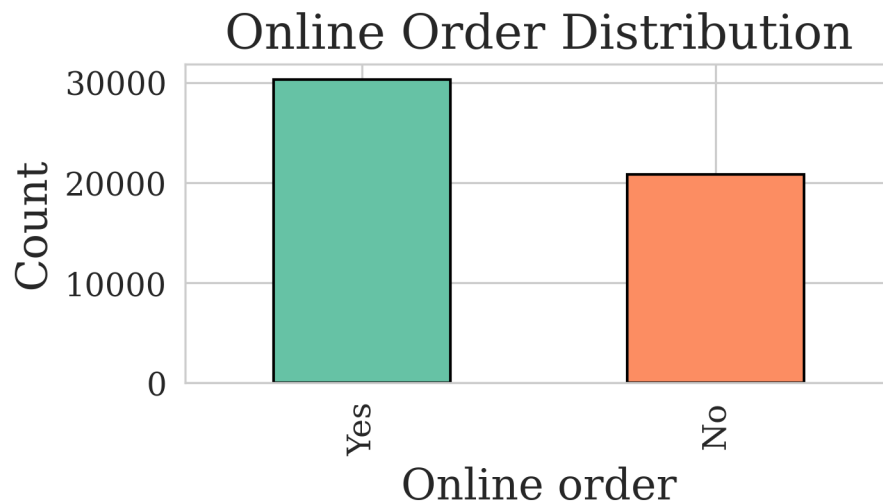
```

# Plot
df['online_order'].value_counts().plot(
    kind='bar',
    color=sns.color_palette('Set2'),
    edgecolor='black'
)

# Labels and title
plt.title('Online Order Distribution', fontsize=18)
plt.xlabel('Online order', fontsize=16)
plt.ylabel('Count', fontsize=16)
plt.tight_layout()

# Show
plt.show()

```



```
[412]: table_counts = df['book_table'].value_counts()
```

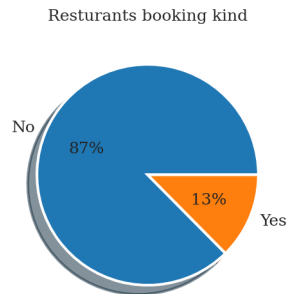
```

[413]: plt.figure(figsize=(2, 2)) # Very small figure

plt.pie(
    table_counts.values,
    labels=table_counts.index,
    textprops={'fontsize': 6}, # Comma added here
    autopct='%1.0f%%',
    shadow=True
)

```

```
plt.title('Resturants booking kind', fontsize=6) # Small title to fit tiny
↪figure
plt.tight_layout()
plt.show()
```



0.0.5 Table booking vs Rate

```
[414]: df['book_table'] == 'Yes'
```

```
[414]: 0      True
      1     False
      2     False
      3     False
      4     False
      ...
      51712  False
      51713  False
      51714  False
      51715   True
      51716  False
      Name: book_table, Length: 51167, dtype: bool
```

```
[415]: df[df['book_table'] == 'Yes'] # Shows the data frame
```

```
[415]:                                     url \
0      https://www.zomato.com/bangalore/jalsa-banasha...
7      https://www.zomato.com/bangalore/onesta-banash...
11     https://www.zomato.com/bangalore/cafe-shuffle-...
12     https://www.zomato.com/bangalore/the-coffee-sh...
44     https://www.zomato.com/bangalore/onesta-banash...
...
51701  https://www.zomato.com/bangalore/the-beer-cafe...
51703  https://www.zomato.com/bangalore/olivers-pub-d...
51704  https://www.zomato.com/bangalore/smaaash-white...
```

```
51705 https://www.zomato.com/bangalore/izakaya-gastr...
51715 https://www.zomato.com/bangalore/chime-sherato...
```

	name	online_order	\
0	Jalsa	Yes	
7	Onesta	Yes	
11	Cafe Shuffle	Yes	
12	The Coffee Shack	Yes	
44	Onesta	Yes	
...	...	...	
51701	The Beer Cafe	Yes	
51703	Oliver's Pub & Diner	Yes	
51704	Smaaash	No	
51705	Izakaya Gastro Pub	Yes	
51715	Chime - Sheraton Grand Bengaluru Whitefield Ho...	No	

	book_table	rate	votes	location	rest_type	\
0	Yes	4.1	775	Banashankari	Casual Dining	
7	Yes	4.6	2556	Banashankari	Others	
11	Yes	4.2	150	Banashankari	Cafe	
12	Yes	4.2	164	Banashankari	Cafe	
44	Yes	4.6	2556	Banashankari	Others	
...	...	...	...	...	...	
51701	Yes	4.1	673	Whitefield	Others	
51703	Yes	3.9	548	Whitefield	Others	
51704	Yes	4.0	189	Whitefield	Others	
51705	Yes	3.8	128	Whitefield	Others	
51715	Yes	4.3	236	Others	Bar	

	approx_cost(for two people)	listed_in(type)
0	800.0	Buffet
7	600.0	Cafes
11	600.0	Cafes
12	500.0	Cafes
44	600.0	Delivery
...	...	...
51701	1400.0	Pubs and bars
51703	1500.0	Pubs and bars
51704	1500.0	Pubs and bars
51705	1200.0	Pubs and bars
51715	2500.0	Pubs and bars

[6449 rows x 10 columns]

```
[416]: # Grab the ratings of the resturant which accepts table booking
np.round(df[df['book_table'] == 'Yes']['rate'].mean(),2)
```

[416]: 4.13

```
[417]: avg_rate_with_booking = np.round(df[df['book_table'] == 'Yes']['rate'].mean(),2)
avg_rate_without_booking = np.round(df[df['book_table'] == 'No']['rate'].
↳mean(),2)
```

```
[418]: avg_rate_with_booking
```

[418]: 4.13

```
[419]: avg_rate_without_booking
```

[419]: 3.64

```
[420]: #You can reset matplotlib settings to default with:
import matplotlib as mpl
mpl.rcParams.update(mpl.rcParamsDefault)
```

```
[421]: df2 = pd.DataFrame.from_dict({
    'Ratings with booking' : [avg_rate_with_booking],
    'Ratings without booking' : [avg_rate_without_booking]

})
df2
```

```
[421]:    Ratings with booking  Ratings without booking
0                4.13                3.64
```

```
[422]: # Reshape your data to long format
df2_melted = df2.melt(var_name='Booking Type', value_name='Average Rating')

# Set figure size
plt.figure(figsize=(8, 8))

# Plot
ax = sns.barplot(
    data=df2_melted,
    x='Booking Type',
    y='Average Rating',
    hue='Booking Type',
    palette='Set2',
    legend=False
)

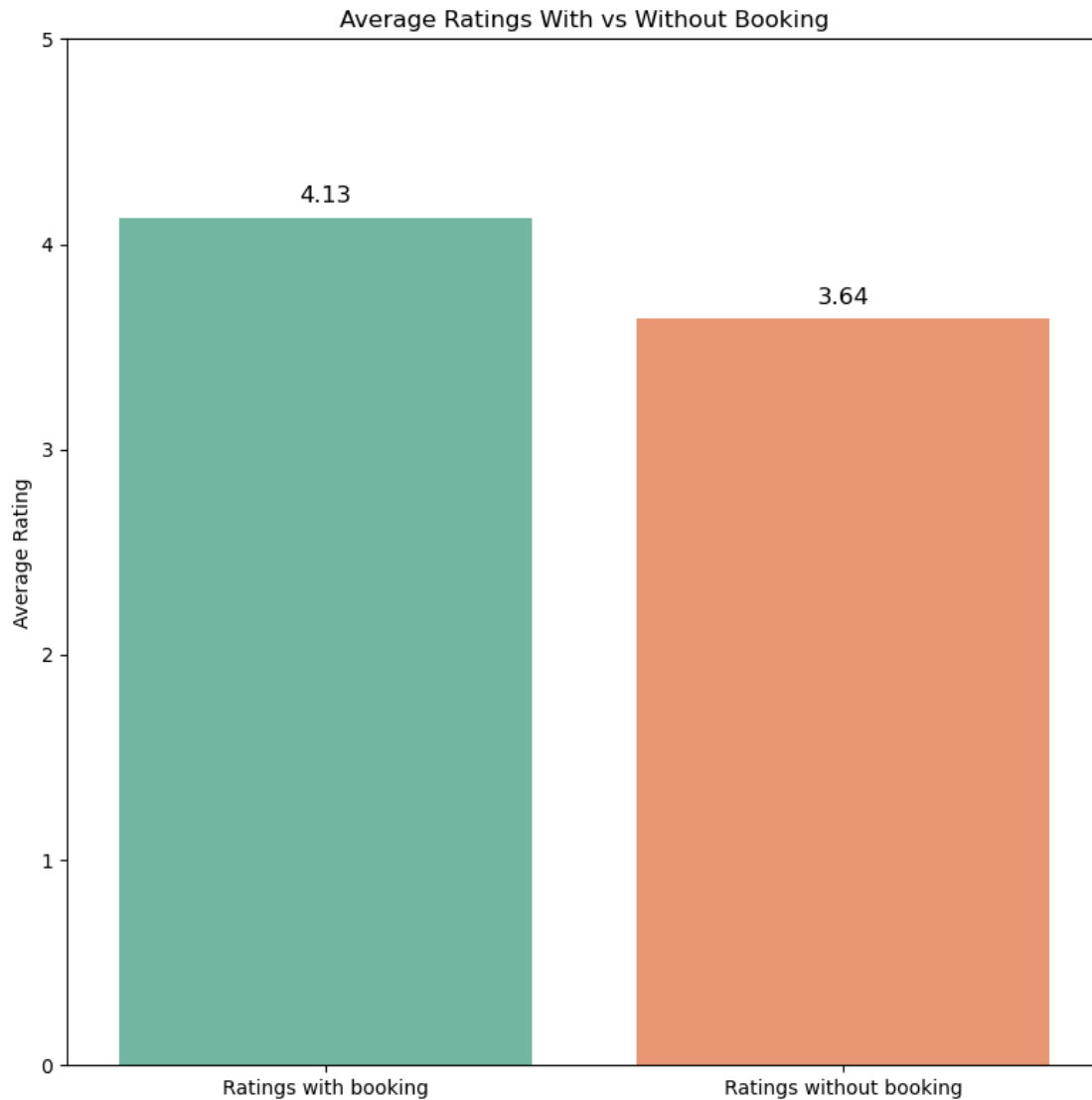
# Add value labels (containers)
for bar in ax.patches:
    height = bar.get_height()
```

```

ax.text(
    bar.get_x() + bar.get_width() / 2,
    height + 0.05, # Slightly above the bar
    f'{height:.2f}',
    ha='center',
    va='bottom',
    fontsize=12
)

# Titles and labels
plt.title('Average Ratings With vs Without Booking')
plt.ylabel('Average Rating')
plt.xlabel('')
plt.ylim(0, 5)
plt.tight_layout()
plt.show()

```

```
[423]: df2_melted
```

```
[423]:
```

	Booking Type	Average Rating
0	Ratings with booking	4.13
1	Ratings without booking	3.64

```
[424]: df.head()
```

```
[424]:
```

	url	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	San Churro Cafe
3	https://www.zomato.com/bangalore/addhuri-udupi...	Addhuri Udupi Bhojana

4 <https://www.zomato.com/bangalore/grand-village...> Grand Village

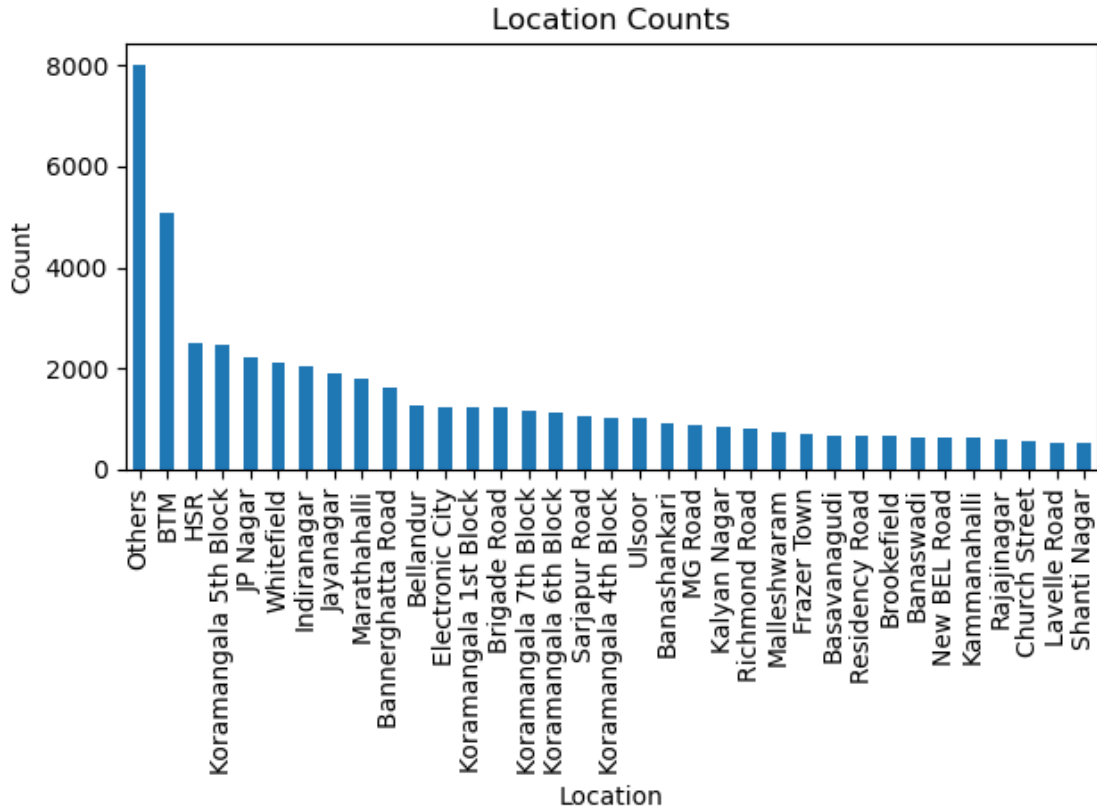
	online_order	book_table	rate	votes	location	rest_type \
0	Yes	Yes	4.1	775	Banashankari	Casual Dining
1	Yes	No	4.1	787	Banashankari	Casual Dining
2	Yes	No	3.8	918	Banashankari	Others
3	No	No	3.7	88	Banashankari	Quick Bites
4	No	No	3.8	166	Basavanagudi	Casual Dining

	approx_cost(for two people)	listed_in(type)
0	800.0	Buffet
1	800.0	Buffet
2	800.0	Buffet
3	300.0	Buffet
4	600.0	Buffet

0.0.6 Location with more number of restaurants

```
[425]: import matplotlib.pyplot as plt

df['location'].value_counts().plot(kind='bar')
plt.title('Location Counts')
plt.ylabel('Count')
plt.xlabel('Location')
plt.tight_layout()
plt.show() # This is what triggers the plot window!
```



```
[426]: df.head()
```

```
[426]:
```

	url	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	San Churro Cafe
3	https://www.zomato.com/bangalore/addhuri-udupi...	Addhuri Udupi Bhojana
4	https://www.zomato.com/bangalore/grand-village...	Grand Village

	online_order	book_table	rate	votes	location	rest_type \
0	Yes	Yes	4.1	775	Banashankari	Casual Dining
1	Yes	No	4.1	787	Banashankari	Casual Dining
2	Yes	No	3.8	918	Banashankari	Others
3	No	No	3.7	88	Banashankari	Quick Bites
4	No	No	3.8	166	Basavanagudi	Casual Dining

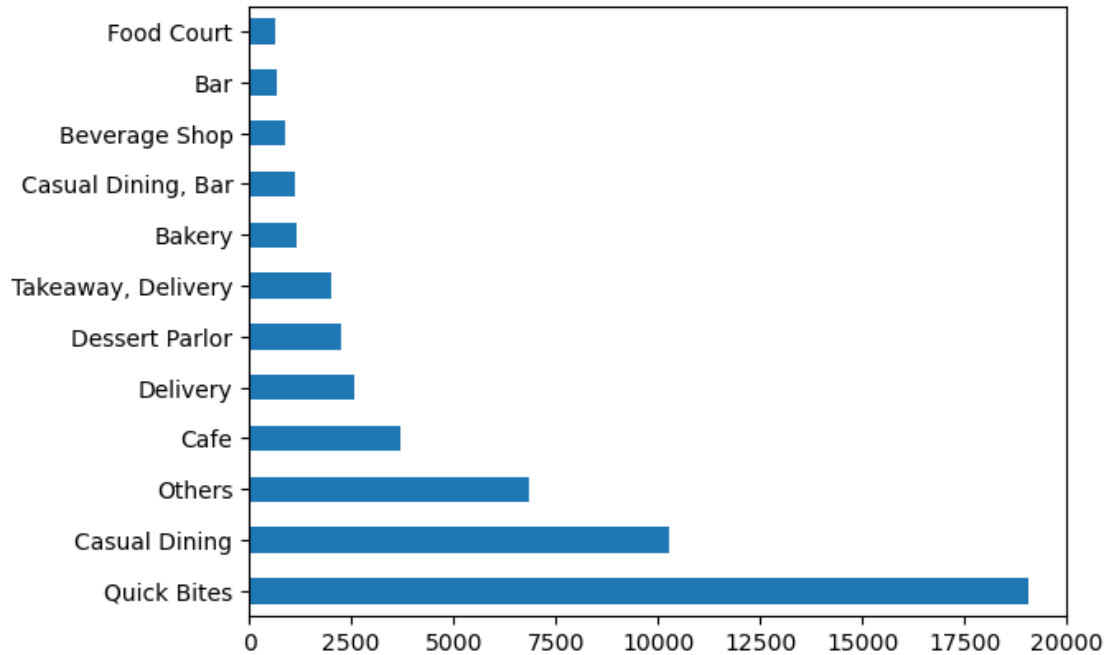
	approx_cost(for two people)	listed_in(type)
0	800.0	Buffet
1	800.0	Buffet
2	800.0	Buffet
3	300.0	Buffet

4

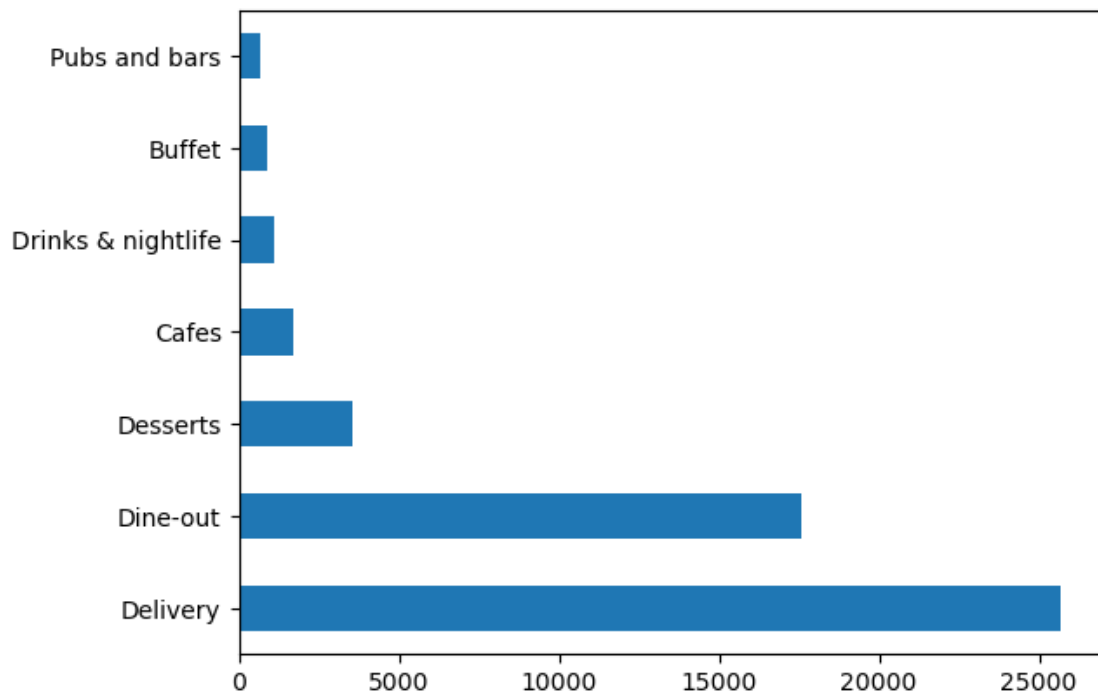
600.0

Buffet

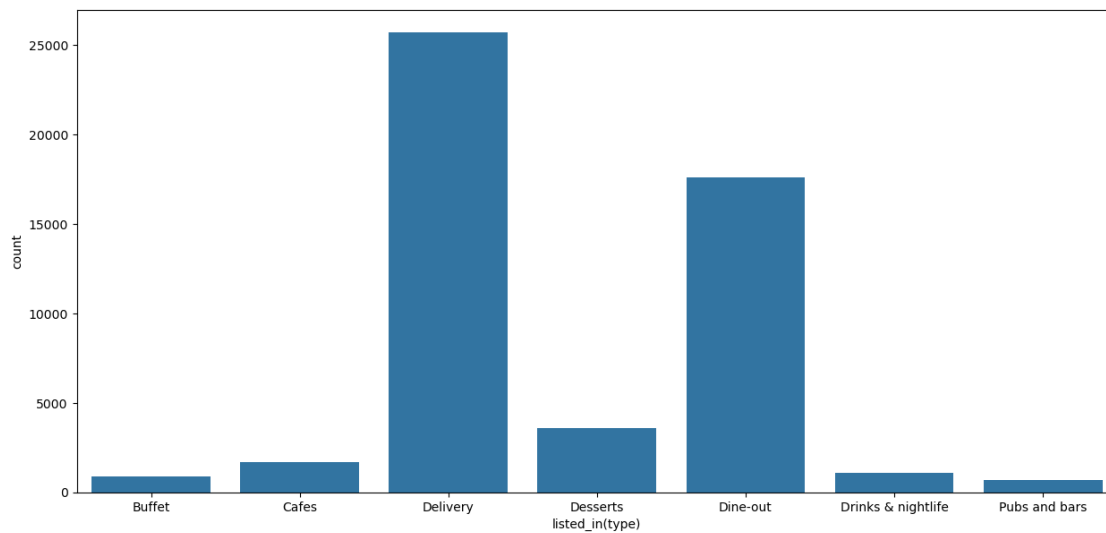
```
[427]: df['rest_type'].value_counts().plot(kind = 'barh')  
plt.show()
```



```
[428]: df['listed_in(type)'].value_counts().plot(kind = 'barh')  
plt.show()
```



```
[429]: plt.figure(figsize = (15,7))  
sns.countplot(x = df['listed_in(type)'])  
plt.show()
```



0.1 Cost of Restaurants:

```
[430]: df['approx_cost(for two people)'].value_counts()
```

```
[430]: 300.0    7549
      400.0    6525
      500.0    4940
      200.0    4846
      600.0    3693
      ...
      560.0     1
      160.0     1
      469.0     1
      3700.0     1
      5000.0     1
      Name: approx_cost(for two people), Length: 70, dtype: int64
```

0.1.1 What is the avg cost of top 10 restaurants based on number of chains in bangalore

```
[451]: # Step 1: Clean cost column (already done)
      #df['approx_cost(for two people)'] = (
      #    df['approx_cost(for two people)']
      #    .astype(str)
      #    .str.replace(',', '')          # Remove comma
      #    .replace('nan', np.nan)
      #    .astype(float)
      #)

      # Step 2: Group cost by restaurant name
      res_cost = df['approx_cost(for two people)'].groupby(df['name'])

      # Step 3: Get top 10 most frequent restaurant names (chosen based on frequency of appearance)
      # Other ways based on ratings, review counts etc.
      top_names = df['name'].value_counts().head(10).index

      # Step 4: Build dictionary with average costs
      d1 = {}
      for name in top_names:
          d1[name] = np.round(res_cost.get_group(name).mean(), 2)

      # Optional: print
      print(d1)
```

```
{'Cafe Coffee Day': 844.79, 'Onesta': 600.0, 'Just Bake': 400.0, 'Empire Restaurant': 685.21, 'Five Star Chicken': 257.86, 'Kanti Sweets': 400.0, 'Petoo': 659.85, 'Polar Bear': 361.54, 'Baskin Robbins': 251.56, 'Pizza Hut': 736.29}
```

```
[452]: cost_df = pd.DataFrame(list(d1.items()), columns = ['Resturant Name',  

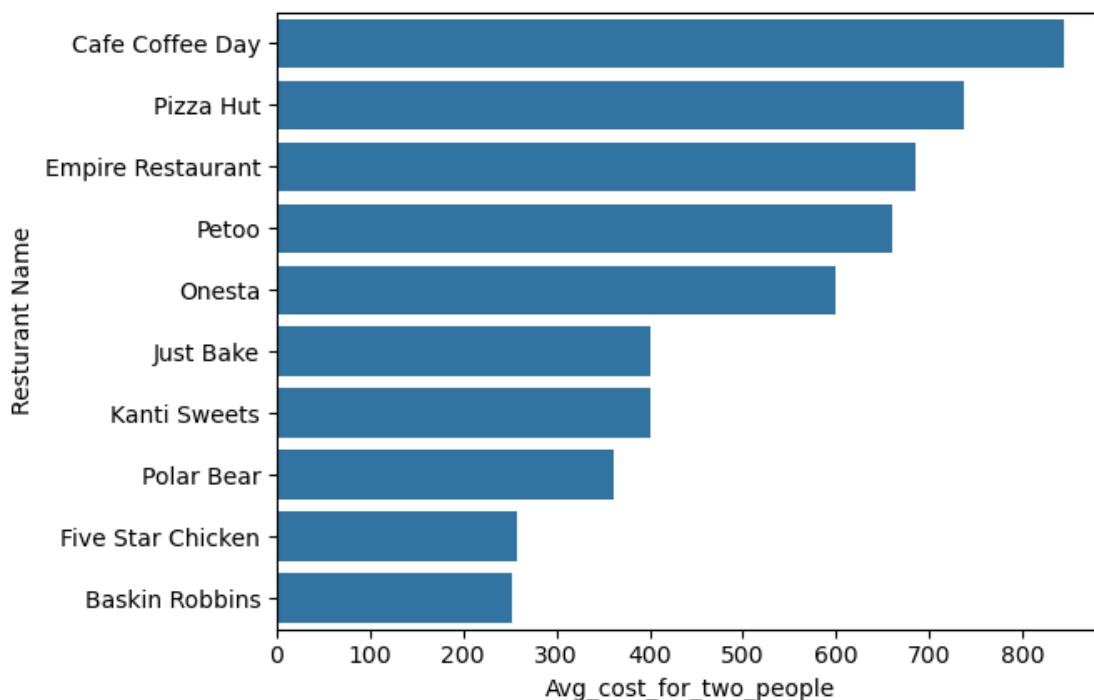
    ↳ 'Avg_cost_for_two_people'] ).sort_values(by = ['Avg_cost_for_two_people'],  

    ↳ ascending = False)
```

```
[453]: cost_df
```

```
[453]:      Resturant Name  Avg_cost_for_two_people
0    Cafe Coffee Day          844.79
9          Pizza Hut          736.29
3  Empire Restaurant          685.21
6           Petoo          659.85
1          Onesta          600.00
2        Just Bake          400.00
5      Kanti Sweets          400.00
7        Polar Bear          361.54
4  Five Star Chicken          257.86
8    Baskin Robbins          251.56
```

```
[454]: sns.barplot(data = cost_df, x = 'Avg_cost_for_two_people', y = 'Resturant Name')
plt.show()
plt.title('This is avg cost for two people in top 10 resturants based on number_
    ↳ of branches')
```



```
[454]: Text(0.5, 1.0, 'This is avg cost for two people in top 10 resturants based on
number of branches')
```

0.1.2 Avg cost based on ratings

```
[455]: import pandas as pd
import numpy as np

'''
# Step 1: Clean the 'approx_cost(for two people)' column
df['approx_cost(for two people)'] = (
    df['approx_cost(for two people)']
    .astype(str)
    .str.replace(',', '')
    .replace('nan', np.nan)
    .astype(float)
) '''

# Step 2: Group by restaurant name and compute mean rating and cost
grouped = df.groupby('name').agg({
    'rate': 'mean',
    'approx_cost(for two people)': 'mean'
}).dropna()

# Step 3: Sort by average rating
top10 = grouped.sort_values(by='rate', ascending=False).head(10)

# Step 4: Round the values for clarity
top10 = top10.round(2)

# Step 5: Rename columns (optional)
top10 = top10.rename(columns={
    'rate': 'Average Rating',
    'approx_cost(for two people)': 'Average Cost for Two'
})

print(top10)
```

name	Average Rating \
Santitas	4.90
Asia Kitchen By Mainland China	4.90
Byg Brewski Brewing Company	4.90
Punjab Grill	4.87
Belgian Waffle Factory	4.84
O.G. Variar & Sons	4.80
Flechazo	4.80

The Pizza Bakery	4.80
AB's - Absolute Barbecues	4.79
Barbecue by Punjab Grill	4.75

	Average Cost for Two
name	
Santã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã Ã...	1000.00
Asia Kitchen By Mainland China	1500.00
Byg Brewski Brewing Company	1600.00
Punjab Grill	2000.00
Belgian Waffle Factory	400.00
O.G. Variar & Sons	200.00
Flechazo	1400.00
The Pizza Bakery	1200.00
AB's - Absolute Barbecues	1568.42
Barbecue by Punjab Grill	1300.00

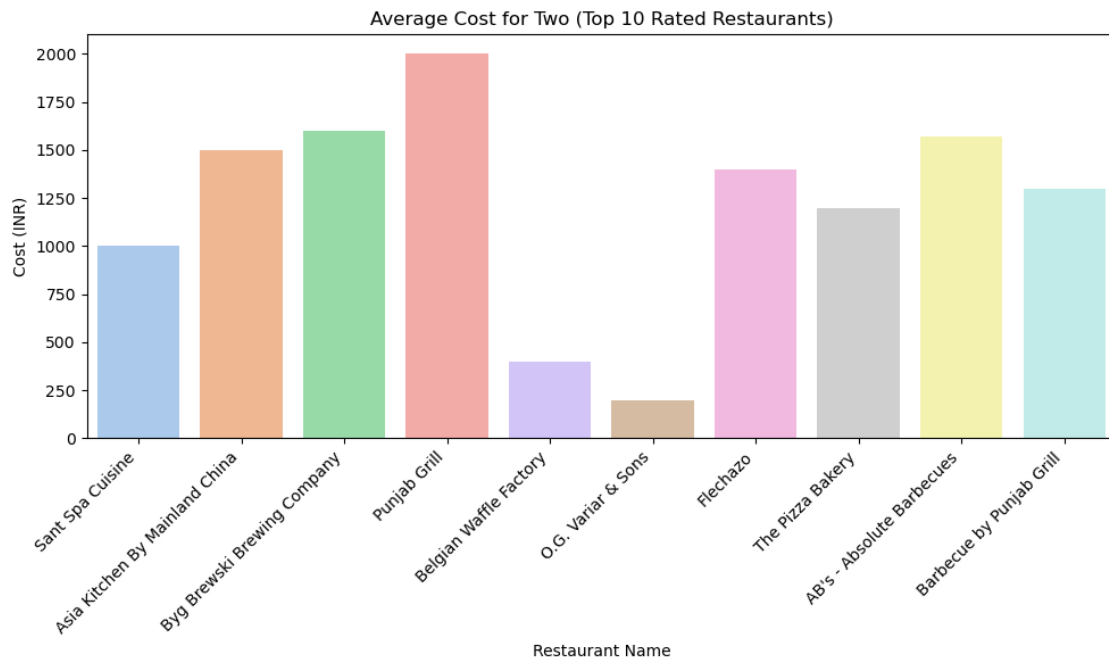
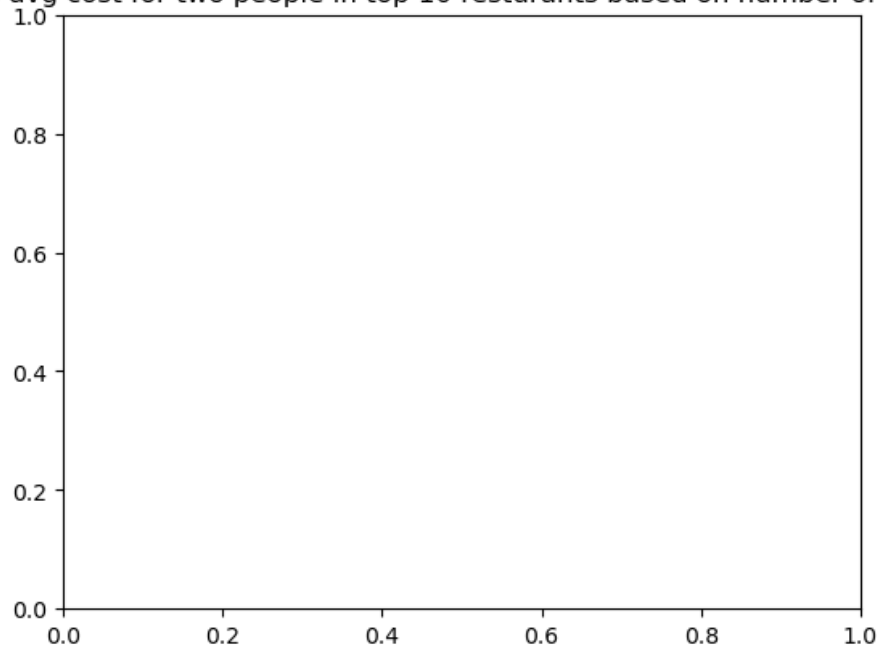
```
[456]: import matplotlib.pyplot as plt
import seaborn as sns

# Optional: Clean non-ASCII characters from restaurant names
top10 = top10.reset_index()
top10['name'] = top10['name'].str.encode('ascii', errors='ignore').str.
    decode('ascii')

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top10,
    x='name',
    y='Average Cost for Two',
    hue='name',          # Avoid FutureWarning
    palette='pastel',
    legend=False         # Prevent duplicated legend
)

plt.title('Average Cost for Two (Top 10 Rated Restaurants)')
plt.xticks(rotation=45, ha='right')
plt.xlabel('Restaurant Name')
plt.ylabel('Cost (INR)')
plt.tight_layout()
plt.show()
```

This is avg cost for two people in top 10 resturants based on number of branches



```
[457]: df.head()
```

```
[457]:
```

	url	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	San Churro Cafe
3	https://www.zomato.com/bangalore/addhuri-udupi...	Addhuri Udupi Bhojana
4	https://www.zomato.com/bangalore/grand-village...	Grand Village

	online_order	book_table	rate	votes	location	rest_type \
0	Yes	Yes	4.1	775	Banashankari	Casual Dining
1	Yes	No	4.1	787	Banashankari	Casual Dining
2	Yes	No	3.8	918	Banashankari	Others
3	No	No	3.7	88	Banashankari	Quick Bites
4	No	No	3.8	166	Basavanagudi	Casual Dining

	approx_cost(for two people)	listed_in(type)
0	800.0	Buffet
1	800.0	Buffet
2	800.0	Buffet
3	300.0	Buffet
4	600.0	Buffet

0.1.3 Top 10 Resturant based o highest avg cost

```
[458]: import pandas as pd
import numpy as np

# Clean the 'approx_cost(for two people)' column if not done already
df['approx_cost(for two people)'] = (
    df['approx_cost(for two people)']
    .astype(str)
    .str.replace(',', '')
    .replace('nan', np.nan)
    .astype(float)
)

# Group by restaurant name and calculate average cost
cost_df = df.groupby('name').agg({
    'approx_cost(for two people)': 'mean'
}).dropna()

# Sort by cost descending and take top 10
top10_cost = cost_df.sort_values(by='approx_cost(for two people)',
    ↪ascending=False).head(10)
top10_cost = top10_cost.round(2)
top10_cost = top10_cost.rename(columns={'approx_cost(for two people)': 'Average_
    ↪Cost for Two'})
```

```

# Optional: Clean non-ASCII characters
top10_cost = top10_cost.reset_index()
top10_cost['name'] = top10_cost['name'].str.encode('ascii', errors='ignore').
↳str.decode('ascii')

# Display the result
print(top10_cost)

```

	name	Average Cost for Two
0	Le Cirque Signature - The Leela Palace	6000.0
1	Royal Afghan - ITC Windsor	5000.0
2	Malties - Radisson Blu	4500.0
3	La Brasserie - Le Meridien	4100.0
4	Jamavar - The Leela Palace	4000.0
5	Dakshin - ITC Windsor	4000.0
6	Edo Restaurant & Bar - ITC Gardenia	4000.0
7	Dum Pukht Jolly Nabobs - ITC Windsor	4000.0
8	Riwaz - The Ritz-Carlton	4000.0
9	Masala Klub - The Taj West End	4000.0

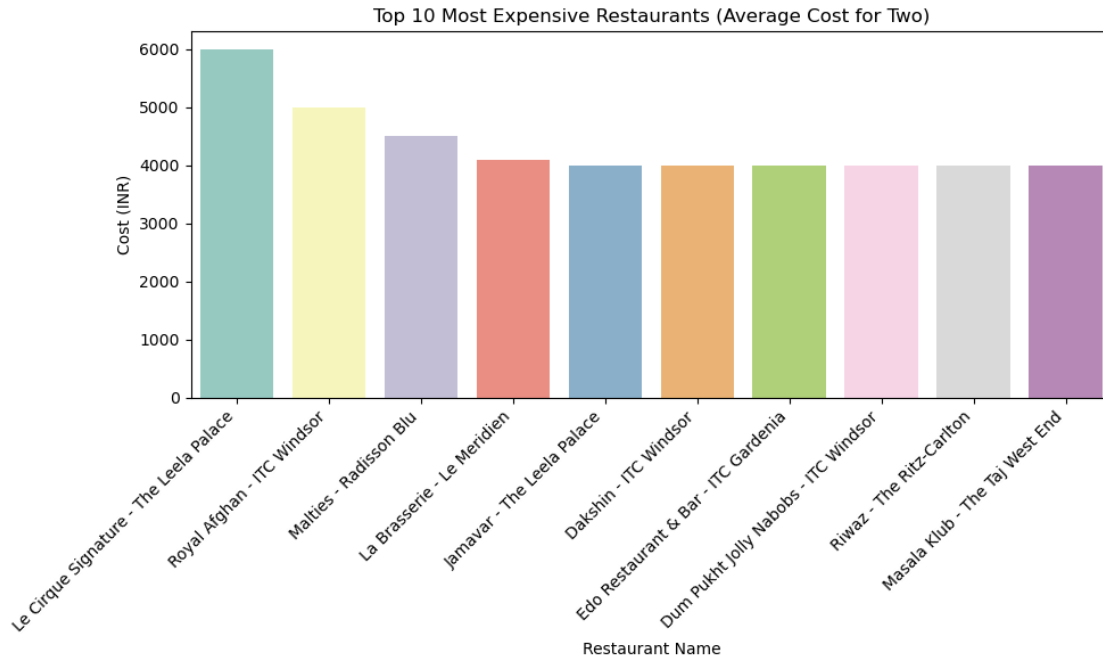
```

[459]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
sns.barplot(
    data=top10_cost,
    x='name',
    y='Average Cost for Two',
    hue='name',
    palette='Set3',
    legend=False
)

plt.title('Top 10 Most Expensive Restaurants (Average Cost for Two)')
plt.xlabel('Restaurant Name')
plt.ylabel('Cost (INR)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```



```
[461]: df.head()
```

```
[461]:
```

	url	name \
0	https://www.zomato.com/bangalore/jalsa-banasha...	Jalsa
1	https://www.zomato.com/bangalore/spice-elephan...	Spice Elephant
2	https://www.zomato.com/SanchurroBangalore?cont...	San Churro Cafe
3	https://www.zomato.com/bangalore/addhuri-udupi...	Addhuri Udupi Bhojana
4	https://www.zomato.com/bangalore/grand-village...	Grand Village

	online_order	book_table	rate	votes	location	rest_type \
0	Yes	Yes	4.1	775	Banashankari	Casual Dining
1	Yes	No	4.1	787	Banashankari	Casual Dining
2	Yes	No	3.8	918	Banashankari	Others
3	No	No	3.7	88	Banashankari	Quick Bites
4	No	No	3.8	166	Basavanagudi	Casual Dining

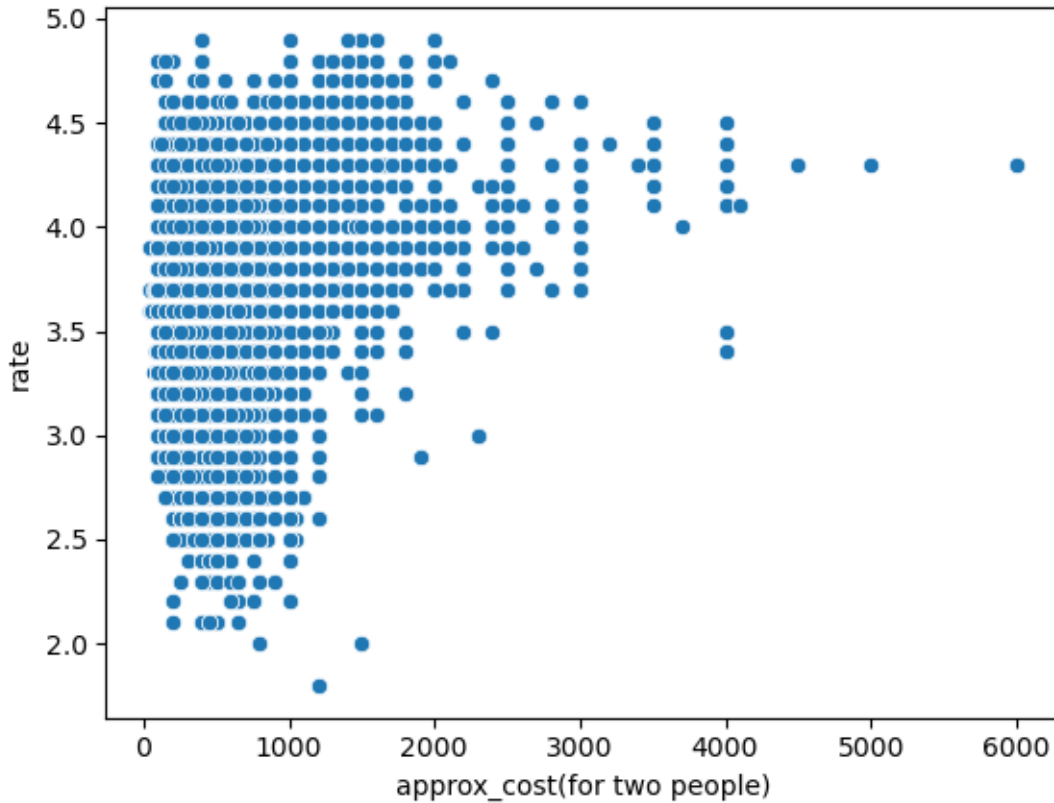
	approx_cost(for two people)	listed_in(type)
0	800.0	Buffet
1	800.0	Buffet
2	800.0	Buffet
3	300.0	Buffet
4	600.0	Buffet

0.1.4 1. Rating vs Cost

- Does expensive food get better ratings?

- Use scatter plot + correlation analysis.

```
[464]: sns.scatterplot(data=df, x='approx_cost(for two people)', y='rate')
plt.show()
```



0.1.5 . Top Locations

- Which locations have the highest average ratings (top 10)?

```
[466]: df.groupby('location')['rate'].mean().sort_values(ascending=False).head(10)
```

```
[466]: location
Lavelle Road          4.106310
Koramangala 5th Block  3.983918
Church Street         3.980316
Koramangala 4th Block  3.880826
Residency Road        3.847478
Koramangala 7th Block  3.837245
MG Road               3.833333
Indiranagar           3.817659
Richmond Road         3.786816
Koramangala 6th Block  3.776882
```

Name: rate, dtype: float64

```
[ ]: 
```

```
[ ]: 
```

```
[ ]: 
```

```
[ ]: 
```

```
[ ]: 
```

```
[ ]: 
```